Nine Blocks, Nine Directions: Weight Tying Turns a Residual Stage into One Smooth Flow

Jaume Ivars Grimalt [1]

Abstract

Residual blocks compute $x + f(x)$, the form of one explicit Euler step. We refine trained residual stages at inference by applying each block R times with step $1/R$, which keeps the integration horizon and needs no retraining. Refined 128-fold, the stages of ConvNeXt, Swin and ResNet lose less than one ImageNet top-1 point, whereas changing the horizon collapses every network. The refined path shows that a trained stage is not one ODE but a composition of one ODE per block. In ConvNeXt-T the field switches at every block boundary, where the heading turns by about $80°$, and each of the nine blocks writes a direction nearly orthogonal to the previous ones (consecutive cosine 0.17, against 0.00 for random blocks of matched strength) that no later block reverses. Sharing one set of weights across the nine blocks turns the stage into a single autonomous flow: it is straighter (0.78 against 0.50), has no switches and is read by its own frozen decoder much earlier, with 72.2% top-1 after three of nine steps against 33.2% for the standard stage. Sharing costs 0.62 points with 33% fewer parameters, and tripling the number of shared steps reaches only 80.26%, recovering none of the gap, which suggests that the gap is the price of a single field rather than of depth. Code: `https://github.com/jaume2000/dynamics_skip_nn`.

1. Introduction

Residual connections (He et al., 2016) made very deep networks trainable by letting each block add a correction to its input, $x \mapsto x + f(x)$. That update has the form of one explicit Euler step of an ODE (E, 2017; Haber & Ruthotto, 2017; Lu et al., 2018), a reading that inspired Neural ODEs (Chen et al., 2018). Whether a *trained* stack behaves as an integrator is debated: Queiruga et al. (2020) already treat residual stages as piecewise-constant in depth and argue that standard ResNets are not meaningful integrators beyond the syntactic analogy; Sander et al. (2022) show that the gap to a Neural ODE need not vanish unless the residual maps vary smoothly in depth; Cohen et al. (2021) find that the deep limit can be an ODE, an SDE or neither; and Marion et al. (2024) prove that a ResNet initialised as a Neural ODE discretisation remains one under gradient flow in the large-depth limit.

We refine trained residual stages at inference and use the refined path as a probe. Write R for the number of applications of each block with step $\Delta t{=}1/R$, so that the integration horizon $T{=}n$ of an n-block stage is kept and nothing is retrained. Convergence itself is not the claim: any Lipschitz residual field has a continuous limit (Appendix D). What matters is that the limit stays usable and that the path is faithful to the trained network. Refined 128-fold, ConvNeXt, Swin and ResNet lose less than one ImageNet top-1 point, whereas changing T collapses every network (Section 3.2): the horizon matters, the resolution does not. On the untied ConvNeXt-T stage, Gram statistics of the block increments agree between $R{=}1$ and $R{=}100$ to within 0.01; on the tied stage the order is kept while the path straightens ($0.72 \to 0.78$). The stage is a composition of one ODE per block, not one ODE with weights smooth in depth: bilinear blending of consecutive block weights costs 6.6 to 23.3 points (Appendix F). The piecewise-constant reading already appears in Queiruga et al.; our novelty is empirical: modern networks already trained, without retraining, and what that flow looks like.

The same probe separates two transports behind the algebra $x + f(x)$ (Figure 1). In ConvNeXt-T the field switches at every block boundary, where the heading turns by about $80°$ ($75°$ to $86°$), and the nine blocks write nearly orthogonal, positively correlated directions (consecutive cosine 0.17, against 0.00 for random blocks of matched strength and 0.92 for the tied stage) that no later block reverses: survival $s_k > 0$ for every image and block, so the opposed fraction $E{=}0$ (Section 4.3). Straightness sits at 0.50, between random (0.33) and tied (0.78); writing $d = \|x_T - x_0\|$, C for the sum of block chords and L for arc length, straightness $= (d/C)\,(C/L)$ with $C/L{=}0.985$, so the detour is made between blocks. Alignment of each increment with the des-

[1]Mycrospace AI Research, Valencia, Spain. Correspondence to: Jaume Ivars Grimalt <jaumeig95@gmail.com>.

arXiv preprint.

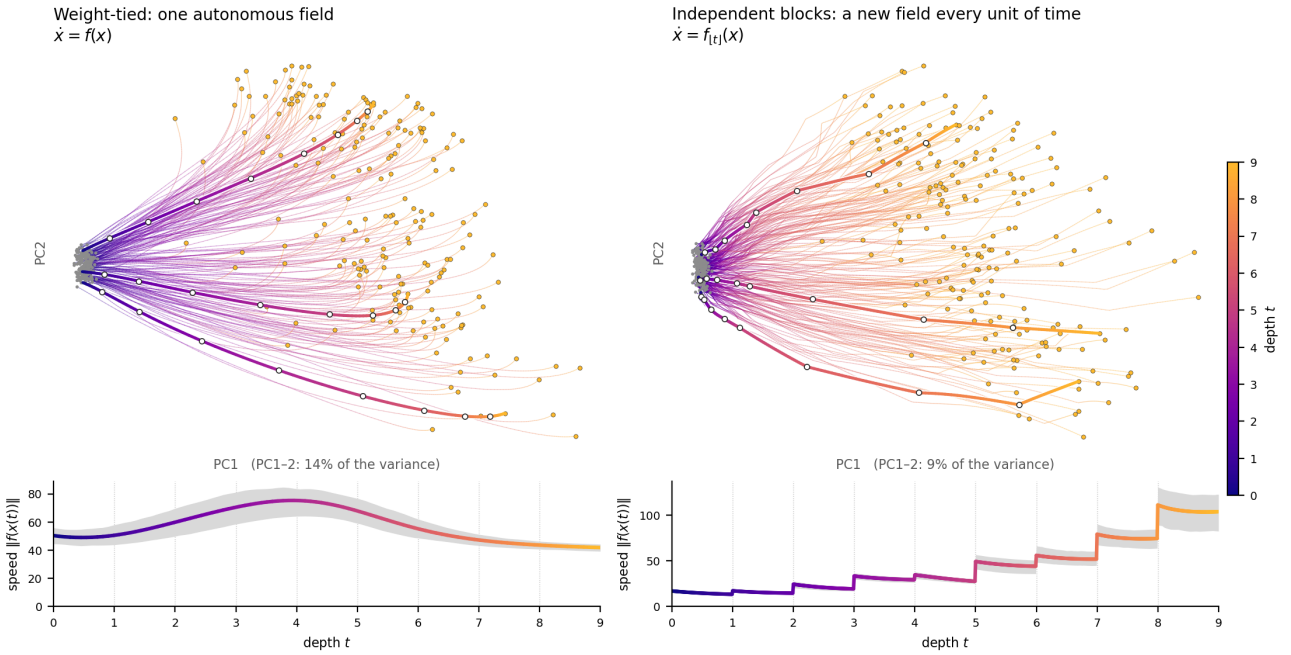


Figure 1. Stage-3 feature trajectories of a weight-tied ConvNeXt (left; $9R$ applications of the shared block with R=100, Δt=0.09) and a standard ConvNeXt-T (right; each of nine blocks applied R=100 times with Δt=0.01), both integrated with Euler up to horizon T=9 (massive channel removed; 200 ImageNet validation images, 20 from each of 10 classes). **Top:** states are flattened and mapped to 2048 dimensions with a fixed Gaussian random projection (Johnson & Lindenstrauss, 1984; Dasgupta & Gupta, 2003), and PCA is fitted on all projected states of each model (Halko et al., 2011); paths coloured by depth t, white dots mark integer t. **Bottom:** residual speed $\|f(x(t))\|$ (mean over images; shade: P10–P90). One shared block is a single autonomous field $\dot{x} = f(x)$; nine independent blocks switch the field at every integer, $\dot{x} = f_{\lfloor t \rfloor}(x)$.

tination rises from 0.15 to 0.69, and the half-progress time is t_{50}=7.45 against 4.19 (tied) and 4.72 (random). This staircase is specific to ConvNeXt: Swin is coherent and the ResNets reverse (Appendix I).

Weight tying turns the stage into one autonomous flow with search, expansion and refinement phases. A frozen decoder reads that state much earlier: native prefixes give 72.2% top-1 at t=3 against 33.2%, and 78.9% against 66.9% at t=5 (Section 5). The frozen decoder is the right measure of early exit without training extra heads; a low score still does not imply that early features lack information. Sharing costs 0.62 points with 33% fewer parameters (single runs, unmatched recipes), and tripling the applications of the shared block (27 applications, same parameters) reaches only 80.26%, recovering none of the gap: the price looks like autonomy, not depth (Sections 4.1 and 5).

Our contributions are:

- **Refinement as a faithful probe** and the switched-flow picture of a trained residual stage (Section 3).
- **Basis-independent transport statistics**, with a random baseline of matched strength and a straightness decomposition into between-block and within-block parts (Section 4.3).
- **Weight tying as an autonomous, anytime flow**, whose accuracy cost is autonomy rather than depth (Sections 4.1 and 5).

2. Related Work

Residual networks as discretised ODEs. A residual update $x + f(x)$ (He et al., 2016) is one explicit Euler step of an ODE (E, 2017; Haber & Ruthotto, 2017; Lu et al., 2018), a reading that led to Neural ODEs (Chen et al., 2018), multi-level training (Chang et al., 2018) and continuous-in-depth architectures (Queiruga et al., 2020). Whether a trained ResNet behaves as such a discretisation is debated (Queiruga et al., 2020; Sander et al., 2022; Cohen et al., 2021; Marion et al., 2024). Those works train continuous models or study growing depth; we refine already trained standard networks at a fixed horizon and use the refined path as a probe. The piecewise-constant-in-depth schedule is already the modelling choice of Queiruga et al. (2020); we take it as given and ask what the flow of modern trained stages looks like.

Weight sharing and looping in depth. Sharing one residual block across depth links ResNets to recurrent networks (Liao & Poggio, 2016); sharing convolutions or soft parameters across blocks yields compact CNNs (Boulch,

2017; Savarese & Maire, 2019). Universal and looped transformers reuse one layer or a block of layers (Dehghani et al., 2019; Giannou et al., 2023), ALBERT shares all layers (Lan et al., 2020) and partial sharing trades parameters against accuracy (Takase & Kiyono, 2023; Bae et al., 2025). Iterating a shared block beyond its training depth degrades unless the model recalls the input and is trained with a random number of iterations (Bansal et al., 2022; Geiping et al., 2025); deep equilibrium models solve for a fixed point instead (Bai et al., 2019). For us weight tying is a control that turns a switched stack into one autonomous field.

Early exit and anytime inference. BranchyNet (Teerapittayanon et al., 2016), multi-scale dense nets (Huang et al., 2018) and shallow-deep networks (Kaya et al., 2019) attach trained intermediate classifiers so that easy examples can leave early. We instead decode prefixes of a weight-tied stage with that stage's own frozen downsample, stage 4 and head: the tied stage is anytime without extra heads. A low early score still does not mean that early features lack information; it means that they are not yet in the region the decoder was trained on.

Iterative refinement and robustness across depth. Residual networks tolerate dropping or reordering blocks, behave like ensembles of shallow paths (Veit et al., 2016; Huang et al., 2016) and have been read as iterative estimators of one representation (Greff et al., 2017; Jastrzębski et al., 2018). The tied stage behaves consistently with that reading; the untied ConvNeXt-T stage does not, which we present as consistent with those works rather than as a proof of them. Middle layers of language models can be skipped or swapped with graceful degradation, although repeating one middle layer is harmful (Sun et al., 2025; Lad et al., 2025), and deep layers can be pruned with light fine-tuning (Gromov et al., 2025).

Geometry of representations along depth. Deep networks gradually untangle and flatten the data manifold (Bengio et al., 2013; DiCarlo & Cox, 2007; Brahma et al., 2016; Hauser & Ray, 2017; Naitzat et al., 2020; Cohen et al., 2020), whose intrinsic dimension changes with depth (Ansuini et al., 2019; Valeriani et al., 2023). Representation trajectories straighten in human perception of videos (Hénaff et al., 2019) and in language models (Hosseini & Fedorenko, 2023), and the residual stream of transformers has been analysed through its velocity, acceleration and turning angles (Fernando & Guitchounts, 2025; Pandey et al., 2026). Those analyses sample one point per layer; refinement gives hundreds per block and separates the motion within blocks from the jumps between them. Centered kernel alignment and related Gram comparisons measure similarity of neural representations (Kornblith et al., 2019); we use the Gram matrix of block *increments* as a basis-independent summary of transport. Outlier features, massive activations in language models (Sun et al., 2024; 2026) and high-norm tokens in vision transformers (Darcet et al., 2024), dominate geometric statistics; we find their analogues in ConvNeXt and Swin and exclude them.

3. Refining Residual Stages

We read a residual vision network as a three-part map: stem, stage 1 and stage 2 encode the image into a latent state x_0; the deepest residual stage transports x_0 to a state x_T; stage 4 and the head decode x_T into logits. We study that middle transport, stage 3 of ConvNeXt (Liu et al., 2022), Swin (Liu et al., 2021) and ResNet (He et al., 2016). It holds most of the residual depth (nine of the eighteen blocks of ConvNeXt-T), keeps one representation and spatial resolution and has enough blocks to study dynamics.

3.1. Setup

Write the residual of block i as $f_i(x) = \mathrm{block}_i(x) - x$. Over a stage of n blocks starting from the output x_0 of the previous stage, the native network takes one Euler step of size 1 per block,

$$x_{i+1} = x_i + f_i(x_i), \qquad x_n = x_0 + \sum_{i=0}^{n-1} f_i(x_i). \quad (1)$$

We take the n blocks as the trained horizon $T{=}n$ and read each block as a field integrated over its own unit of depth, so that the continuous object behind (1) is

$$x_{i+1} = x_i + \int_i^{i+1} f_i\big(x(t)\big)\, dt, \qquad T = n, \quad (2)$$

a switched system in the control sense (Liberzon, 2003), with the piecewise-constant schedule $\theta_t = \theta_{\lfloor t \rfloor}$. Per-block step sizes are absorbed into the learned weights, so integrating each block over one unit of depth matches the step the model has internalised. In a weight-tied stage $f_i \equiv f$ and (2) collapses to a single autonomous ODE,

$$x_T = x_0 + \int_0^T f\big(x(t)\big)\, dt, \qquad T = n. \quad (3)$$

Repetitions R. We refine with one parameter R: each block is applied R times with step $\Delta t{=}1/R$, so that $T{=}nR\,\Delta t{=}n$ is kept and only the resolution changes,

$$x_T = x_0 + \sum_{i=0}^{n-1} \sum_{r=0}^{R-1} \Delta t\, f_i\big(x_{i+r\Delta t}\big). \quad (4)$$

For a weight-tied stage R means nR applications of the shared block. $R{=}1$ recovers the native network; as R grows the scheme approaches (2), and we also compare

Midpoint (RK2) and RK4 on the same horizon. Nothing is retrained. Refinement converges for any Lipschitz residual field, trained or random (Appendix D), so the question is what the limit keeps and how the path is organised.

3.2. The limit is usable; the horizon matters, not the resolution

We refine our ConvNeXt-T and weight-tied ConvNeXt-T, and the torchvision Swin-T (`IMAGENET1K_V1`), ResNet-50 and ResNet-101 (`IMAGENET1K_V2`) (Figure 2, left; Table 1). All models are evaluated on the full validation set with one transform (resize to 256, bilinear, centre crop 224), so the torchvision models score slightly differently from their reported 81.47, 80.86 and 81.89%, which use their own resize. Accuracy decreases monotonically with R and plateaus at a limit that Euler, Midpoint and RK4 share once the step is fine enough (Appendix B); the endpoints themselves agree to within $0.25°$ (Appendix C). The limit costs 0.22 to 0.91 points. The native network is the best point of every sweep and is *not* at the limit: RK4 with the nine native unit steps already lands on it. The decoder is tuned to the state produced by one Euler step of size 1, not to the flow that step approximates.

Repeating blocks without shrinking the step (R=10 or 100 with Δt=1) collapses every model to 0.10–1.03% top-1, and the weight-tied network collapses in the same way when T leaves the trained horizon (Appendix B), as recurrent networks iterated past their training depth do (Bansal et al., 2022). Even R=3 with Δt=1 (T=27) drops ConvNeXt-T to 41.0%. With large steps at T=9 the tied model needs a higher-order solver: with 4 steps, RK4 reaches 78.9% and Euler 67.8%. Those coarse steps evidence the horizon; they are not a general compute advantage of the tied stage (Appendix B).

3.3. Fidelity and the switched picture

Near-baseline accuracy does not mean that R steps of size $1/R$ reproduce the unit step. On ConvNeXt-T the refined endpoint (R=100) sits $21°$ from the native one, with a relative distance of 0.36 while the norms differ by 3%; on the weight-tied model the gap is $11°$ and 0.20 (Appendix C; Figure 2, right). The untied gap accumulates block after block, whereas the tied one stops growing after two or three steps. Stage 4 and the head nevertheless classify the refined state at a small cost.

The refined path is faithful to the native network in the statistics that matter for transport. On the untied stage, straightness and consecutive cosine at R=1 and R=100 agree to within 0.01 (Table 3). On the tied stage the order of the three regimes is preserved while the path straightens ($0.72 \to 0.78$), as a single autonomous field should under refinement. Together with the usable limit, this is the sense in which the probe is faithful: the horizon is kept, the decoder still reads the state, and the Gram geometry of the trained stage is recovered.

Table 1. Euler refinement with $\Delta t = 1/R$ (T kept), ImageNet top-1 (%). For the weight-tied model R means $9R$ applications of the shared block. ODE limit: RK4 at R=128 (tied: 1024 steps).

R (Δt)	CNX-T	Swin-T	R-50	R-101	Tied
1 (1)	80.96	81.08	80.34	81.67	80.34
2 (0.5)	80.78	80.90	80.06	81.58	80.27
4 (0.25)	80.52	80.65	79.84	81.38	80.21
10 (0.1)	80.36	80.47	79.61	81.22	80.13
32 (0.03)	80.29	80.38	79.50	81.12	80.11
128 (0.008)	80.24	80.34	79.45	81.06	80.12
ODE limit	80.23	80.32	79.43	81.05	80.12
Gap (R=1 $\to$ODE)	−0.73	−0.76	−0.91	−0.62	−0.22

The same measurements give the switched picture. The residual norm of an untied stage jumps at every block boundary (Figure 2, right), and replacing the piecewise-constant schedule by a bilinear blend of consecutive block weights costs 6.6 to 23.3 points on every architecture (Appendix F). Each trained block is meant for its own unit of depth: a stage is a composition of one ODE per block, not one ODE with weights smooth in depth.

4. One Flow or Nine Directions

4.1. The weight-tied control

To separate what depends on having nine independent blocks from what depends only on the residual form, we train a ConvNeXt-T whose stage 3 applies one shared block nine times. Reusing blocks of a model that was trained with distinct blocks tests a different object, and fails (Appendix E); here the stage is *trained* to use a single function. Both models are trained by us on ImageNet-1k for 300 epochs with the same augmentation, regularisation, optimiser and effective batch size of 1024. The runs are single runs with unmatched recipes: the untied model uses peak learning rate 5×10^{-4} with a 20-epoch warmup on a single process, the tied one 1×10^{-3} with a 10-epoch warmup on four distributed processes; no run fixes a seed (Appendix A). The accuracy gap below is therefore indicative, not a controlled ablation.

The untied ConvNeXt-T reaches 80.96% top-1 with 28.59M parameters, below the 82.1% of the original recipe (Liu et al., 2022), which uses a batch of 4096. The weight-tied model reaches 80.34% with 18.97M parameters (33% fewer, −0.62 points), and sharing one block per stage in all four stages reaches 78.99% with 8.68M (70% fewer, −1.97 points). Tripling the applications of the same shared block, still with the tied parameter count, yields a stage of 27 unit

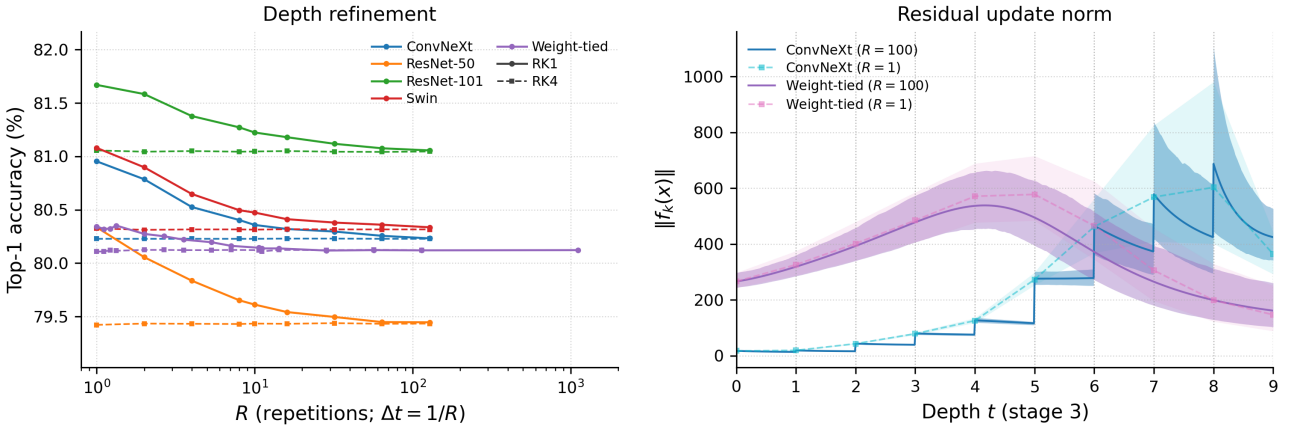


Figure 2. **Left:** top-1 against repetitions R (solid: Euler; dashed: RK4), with $\Delta t{=}1/R$ so that $T{=}n$ is kept (weight-tied: nR applications of the shared block). **Right:** residual norm $\|f(x)\|$ along stage 3 (mean and P5–P95 over 500 images). The untied field is piecewise constant in depth; the weight-tied model is a single smooth autonomous field.

steps trained from scratch at $T{=}27$, $\Delta t{=}1$ (Appendix A). That run reaches 80.26% top-1, -0.08 points relative to the 9-step tied stage, so it recovers none of the 0.62-point gap to the untied model. One residual function applied nine times is thus close to nine independent ones, and extra depth with the same field does not close what remains. The tied stage is a clean autonomous ODE $\dot{x} = f(x)$ over $T{=}9$, and we use it as the control against which the switched stack is read.

4.2. Kinematics

Massive activations. In both ConvNeXt models one stage-3 channel exceeds all others by more than an order of magnitude (channel 236 tied, 195 untied), and in Swin the four corner tokens do (Sun et al., 2024; Darcet et al., 2024); ResNet shows no such outlier. The network relies on the ConvNeXt channel (zeroing it in the tied model costs 12.5 points, a random channel at most 0.3), but it would dominate every geometric statistic, so we remove it from all measurements and leave the forward pass unchanged (Appendix G).

Kinematics. On refined trajectories ($R{=}100$, $\Delta t{=}0.01$, $N{=}500$) we measure the speed $\|v\|$, the angular velocity ω of the heading $\hat{v}$, the split of the acceleration into a tangential part $a_t = \langle \hat{v}, a \rangle$ and a normal part a_n and the curvature $\kappa = a_n/\|v\|^2$ (Figure 3 and Table 2). Curvature is read against the arc travelled per unit of depth, $s \approx \|v\|\Delta t$: the heading turns by $\Delta\theta \approx \kappa\|v\|\Delta t$ (equivalently $\omega\Delta t$). A value $\kappa \sim 10^{-2}$ with $\|v\|$ of tens still turns the heading by tens of degrees over a block. Earlier work measures turning angles between consecutive layers or frames (Hénaff et al., 2019; Hosseini & Fedorenko, 2023; Pandey et al., 2026); refinement gives them within a block.

The tied stage searches, expands and refines. The shared field traces one arc in three phases (Figure 3, top). Early on the speed is moderate while ω and κ peak ($0.70\,\mathrm{rad}/t$ at $t{=}0.58$), so the heading turns by about $40°$ per unit of depth: small steps that keep changing direction. The tangential acceleration then turns positive, peaks at $t{=}2.34$ and crosses zero at $t{=}3.98$, where the speed peaks; κ reaches its minimum just before ($4.2{\times}10^{-3}$ at $t{=}3.79$, about $18°$ per unit of depth). The tangent still carries 28% of $\|a\|$, a large share in a 75,000-dimensional state, but a_n never vanishes. After mid-horizon the field brakes (a_t is most negative at $t{=}5.35$), the speed falls and ω rebounds near $t{=}5.8$. At $t{=}9$ the speed is still 41.7 with $\omega{=}0.20$: the state is coasting, not at a fixed point, which is consistent with the collapse when the integration continues past T (Section 3.2).

Table 2. Stage-3 kinematics ($R{=}100$, $N{=}500$, massive channel removed). Untied values inside blocks; sign flips: share of coordinates of v that change sign across one micro-step.

	Tied	Untied
$\|v\|$	peak 76.2 at $t{=}3.98$	14.8 → 112.5 in steps
ω (rad/t)	peak 0.70 at $t{=}0.58$	0.64–0.87 → 0.46–0.56
κ (10^{-3})	14.3, min 4.2 at $t{=}3.79$	44.4 (block 0), 6.0 (block 8)
$a_n/\|a\|$	0.95	0.97
$\|a\|$, first→last	34.3 → 8.4	14.5 → 53.5
turn at boundary	–	75–86°
sign flips, switch	–	44.6%
sign flips, inside	0.12%	0.19%

The untied stage is a switched staircase. The untied ConvNeXt-T has no such arc (Figure 3, bottom). At each integer the next block replaces the field and the heading jumps by 75° to 86° (80° on average) in a single micro-step. The large ω spikes at those boundaries are not turns of a hundred radians: they are $\arccos\langle \hat{v}_d, \hat{v}_{d+1} \rangle / \Delta t$ on one micro-step of $\Delta t{=}0.01$, so the heading jump is $\omega\Delta t \approx 1.3$–$1.5\,\mathrm{rad}$ (75°–86°). Dividing a finite angle by a vanishing step is

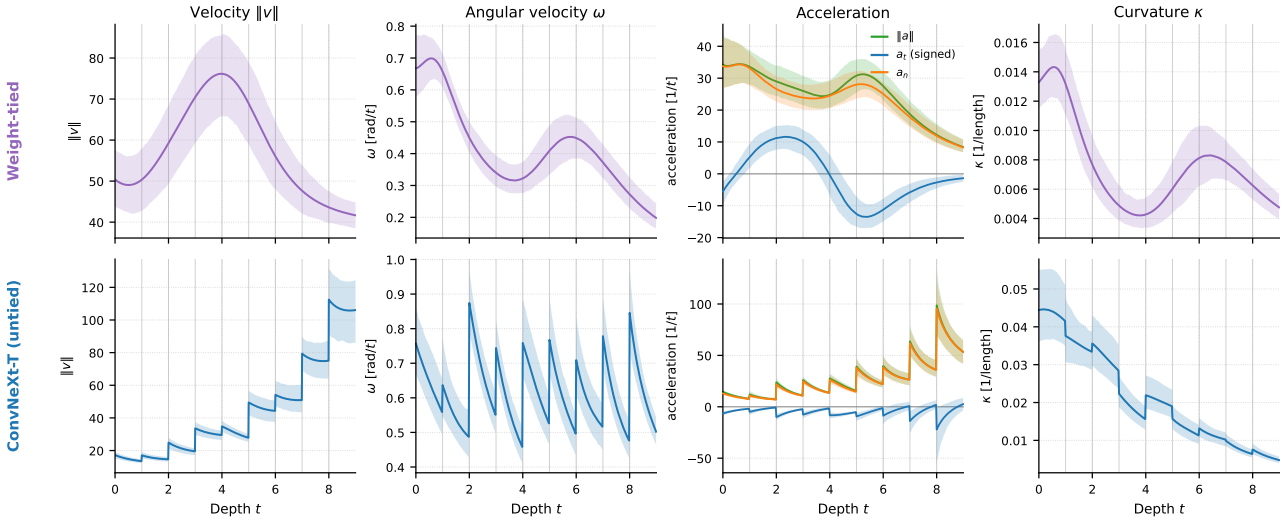


Figure 3. Stage-3 kinematics over T=9 (R=100, Euler Δt=0.01; massive channel removed; mean and P5–P95 over N=500). **Top:** weight-tied ConvNeXt. **Bottom:** untied ConvNeXt-T. Columns: speed $\|v\|$, angular velocity ω, acceleration $(\|a\|, a_t, a_n)$, curvature $\kappa{=}a_n/\|v\|^2$. On the untied row, pairs of micro-steps that straddle a block boundary are omitted from ω, a and κ: there v is discontinuous and finite differences diverge as $\Delta t \to 0$.

why the spikes explode; they only record that neighbouring blocks do not share a direction. At the switch, 44.6% of the coordinates of v change sign, against 0.19% per micro-step inside a block and 0.12% in the tied model, which has no boundaries. Inside a block the motion is short: ω starts at 0.64–0.87 and settles at 0.46–0.56, and κ falls from $4.4{\times}10^{-2}$ in the first block to $6.0{\times}10^{-3}$ in the last as the speed grows. The speed envelope rises in steps, from 14.8 in the first block to 112.5 at the start of the last, with no stage-wide acceleration and braking. The path is therefore a chain of nine shallow arcs joined by near-orthogonal turns; Section 4.3 measures how these pieces add up.

4.3. Nine directions

The untied heading turns by 80° at every boundary. That alone does not say whether the nine blocks cooperate, ignore each other or work against each other, and per-coordinate statistics cannot settle it, because the feature basis is not privileged: rotating the channels changes them while leaving the network's function intact (Appendix H). We therefore use only quantities that depend on the Gram matrix of the block increments, $G_{jk} = \langle \Delta x_j, \Delta x_k \rangle$ with $\Delta x_k = x(k{+}1) - x(k)$, which is invariant to any orthogonal change of basis. We compare three stages on the same 500 validation images (one image from each of 500 classes drawn at random, seed 0; massive channel removed): the weight-tied and untied ConvNeXt-T, and a *random* untied stage with Xavier weights whose gain $g^*{=}1.35$ matches the per-block field strength $\|\Delta x_k\|/\|x_k\|$ of the trained untied model. Table 3 collects the statistics and Figure 4 shows them per block.

Table 3. Basis-independent transport statistics of stage 3, mean over 500 images (std across images ≤ 0.021 for straightness and ≤ 0.035 for cosines). Str.: $\|x_T - x_0\|$ over path length (arc at R=100, chords at R=1); chords: $\|x_T - x_0\|$ over the sum of block chords; $\overline{\cos}$: mean off-diagonal cosine; E: opposed fraction; align: mean $\cos(\Delta x_k, x_T - x_0)$; t_{50}: first t with $p(t) \geq 0.5$.

	Str.	chords	$\cos_{k,k+1}$	$\cos_{k,k+2}$	$\overline{\cos}$	min s_k	E	align	t_{50}
Random R=1	0.330	0.330	0.001	0.002	−0.002	−0.18	0.022	0.330	–
Random R=100	0.324	0.329	−0.001	−0.001	−0.004	−0.19	0.026	0.328	4.72
Untied R=1	0.501	0.501	0.165	0.136	0.093	0.11	0	0.393	–
Untied R=100	0.502	0.510	0.171	0.143	0.099	0.13	0	0.394	7.45
Tied R=1	0.718	0.718	0.768	0.579	0.408	0.57	0	0.685	–
Tied R=100	0.779	0.785	0.922	0.746	0.527	0.84	0	0.757	4.19

Untied blocks write nearly orthogonal directions. Random residual blocks of matched strength write mutually orthogonal increments: every off-diagonal cosine is 0.00 (Figure 4, top left). The trained untied blocks are not random: consecutive increments have cosine 0.17 and increments two blocks apart 0.14, all positive. They are still far from the tied field, where consecutive increments have cosine 0.92 and remain correlated across the whole stage. Each untied block therefore writes mostly in a new direction, slightly biased toward the directions of its neighbours.

No block is erased. The survival $s_k = \sum_{j>k} G_{kj}/G_{kk}$ measures what later blocks do along the direction of block k: $s_k > 0$ means they continue it, $s_k < 0$ that they move against it and $s_k \leq -1$ that they remove it entirely. In the untied model s_k is positive for every image and every block (minimum 0.13), so the opposed fraction E, the mass-weighted share of increments that later blocks reverse, is exactly 0. Random blocks, by contrast, have $s_k < 0$ in 61% of the image and block pairs. The untied stage is thus a

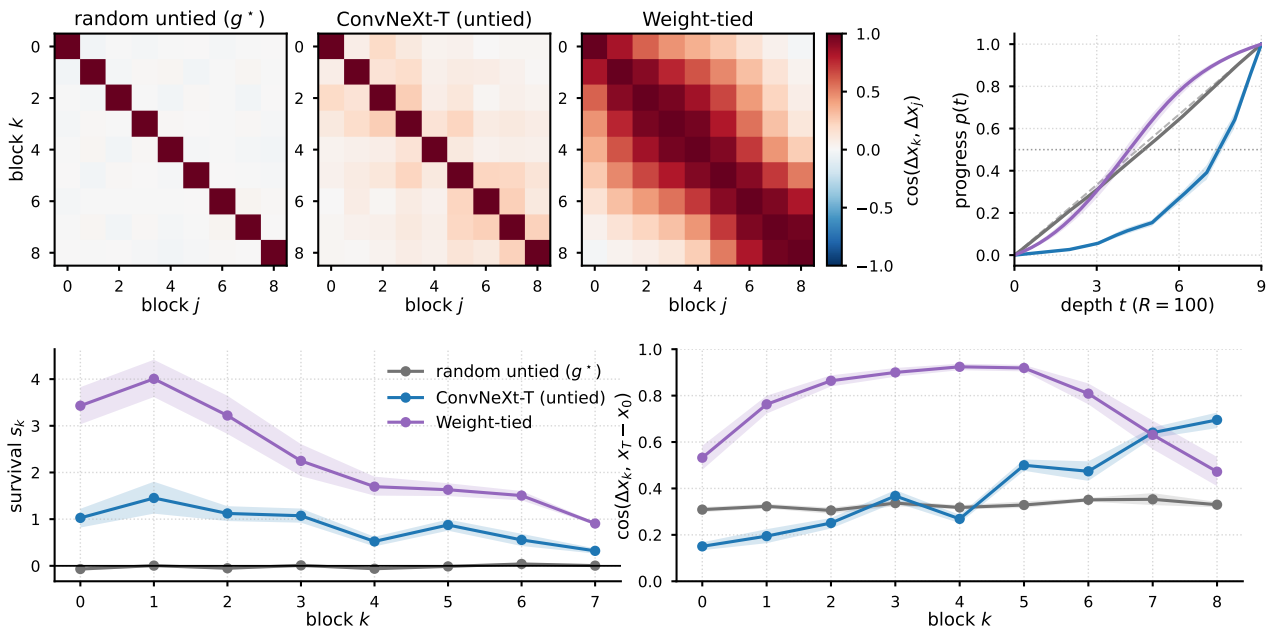


Figure 4. Basis-independent transport statistics of stage 3 (R=100, N=500, massive channel removed). **Top left:** mean cosine between block increments Δx_j and Δx_k for a random untied stage of matched strength, the untied ConvNeXt-T and the weight-tied model. **Top right:** progress $p(t)$ along the chord $x_T - x_0$ (dashed: diagonal). **Bottom left:** survival s_k, the net projection of all later increments on Δx_k in units of $\|\Delta x_k\|^2$ ($s_k < 0$: later blocks oppose block k). **Bottom right:** alignment of each increment with the total displacement. Bands: P10–P90 over images.

staircase: each block adds a new, mostly orthogonal step that no later block takes back.

A straightness ladder. Write $d = \|x_T - x_0\|$, $C = \sum_k \|\Delta x_k\|$ for the sum of block chords and L for the arc length of the refined path. Straightness decomposes as

$$\frac{d}{L} = \frac{d}{C}\frac{C}{L}. \tag{5}$$

The factor d/C is the between-block contribution (how aligned the chords are); C/L is the within-block contribution (how much each block bends relative to its chord). Straightness orders the three stages: 0.33 for random blocks, 0.50 for untied blocks and 0.72 to 0.78 for the tied field. Nine orthogonal steps of equal length would give $1/\sqrt{9}$=0.33, which is where the random stage sits. For the untied stage C/L=0.985, so the arc is only 1.5% longer than the sum of block chords; random blocks turn by a similar amount (0.987) and the tied field by less (0.993). The detour of the untied stage is made between blocks, not within them.

Per-coordinate sign changes between consecutive blocks ($\approx 45\%$) are what any basis gives under a consecutive cosine of 0.17 (Appendix H); they are not evidence that some blocks reverse others.

Alignment with the destination. Projecting the path on the chord $x_T - x_0$ shows when progress is made (Figure 4, top right). The untied stage crosses half of its net displacement at t_{50}=7.45, the tied one at 4.19 and the random one at 4.72, close to the linear 4.5. Per block, the alignment $\cos(\Delta x_k, x_T - x_0)$ of the untied increments rises from 0.15 in the first block to 0.69 in the last, while the random stage stays at 0.33 and the tied field peaks at 0.92 mid-stage (Figure 4, bottom right). Early untied blocks write directions that are almost orthogonal to where the stage ends; the last blocks write most of the final displacement.

Other architectures. The staircase is not universal (Appendix I). Swin-T stage 3 is coherent, with straightness 0.80 (0.67 without the corner tokens) against 0.42 for random blocks and positive survival everywhere. ResNet-50 and ResNet-101, whose stage-3 residual includes the ReLU after the sum, go the other way: straightness 0.37 and 0.16, below their random baselines of 0.43 and 0.22 to 0.23, with genuine reversal (E of 0.19 and 0.24 against about 0.07–0.09 for their random baselines). We have not tested whether the post-sum ReLU explains that reversal.

5. What the Smooth Flow Buys and What It Costs

Anytime reading with the frozen decoder. After a prefix of stage 3 we skip the rest of the stage and classify with the model's own trained downsample, stage 4 and head (Figures 5 and 6; full validation). No probe is trained (Alain & Bengio, 2017; Belrose et al., 2023), so the score measures how close each prefix is to the state the decoder expects. On native prefixes, top-1 at t=3 is 72.2% for the tied stage against 33.2% for the untied one; at t=5 it is 78.9% against 66.9%. Refining the path at R=20 moves the same curves only slightly (74.7% and 30.6% at t=3). The decoder was trained on x_T, so a low score does not mean that early features lack information; it means that the tied state enters the decoder's operating region earlier, as the alignment and t_{50} of Section 4.3 suggest. Alignment and t_{50} are consistent with this mechanism; they do not prove it causally. The frozen decoder is the right measure of early exit without training extra heads: the tied stage is anytime with its own downstream network.

Figure 6 puts the same prefixes on a compute axis against other ways of spending few block evaluations at horizon T=9. At three evaluations the tied prefix already reaches 72.2%, while the equispaced untied triple $\{0, 4, 8\}$ with Δt=3 reaches only 60.6% and the best two-block pair we tested reaches 57.6% (Appendix E). Equispaced untied subsets climb toward the native score as L grows (e.g. 73.01% at L=4, 78.25% at L=6, 80.53% at L=8), but they stay below the tied prefix until late in the stage. Coarse Euler on the tied field at the same T is worse still at small step counts (37.5% with three steps of size 3), so enlarging Δt is not a substitute for the anytime prefixes.

What it costs. Sharing costs 0.62 points with 33% fewer parameters (Section 4.1). Tripling the applications of the shared block, at the same parameter count, reaches only 80.26% and recovers none of the gap (-0.08 points relative to the 9-step tied stage). A reading, as a hypothesis rather than a fact: the gap is not bought by depth but by having distinct fields; it is the price of autonomy, because a single field cannot depend on t. That reading connects to the conjecture that the massive channel acts as an implicit clock inside an autonomous ODE (Appendix G). A natural test, not yet run, is a shared block conditioned on t: if the gap closes, the missing ingredient was non-autonomy rather than capacity.

6. Discussion

Table 4 collects the measurements that carry our reading. The same residual algebra $x + f(x)$ supports two different transports, and refinement at fixed horizon makes the difference visible without retraining either model.

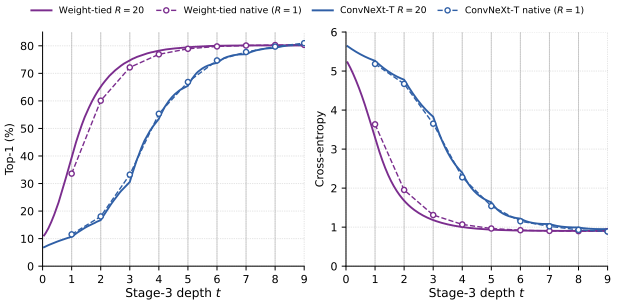


Figure 5. Top-1 (left) and cross-entropy (right) of each model's frozen stage 4 and head on stage-3 prefixes (full validation). Solid: refined path (R=20, Δt=0.05); open markers: native prefixes at block ends. The tied stage is readable much earlier; a low early score does not imply that early features lack information.

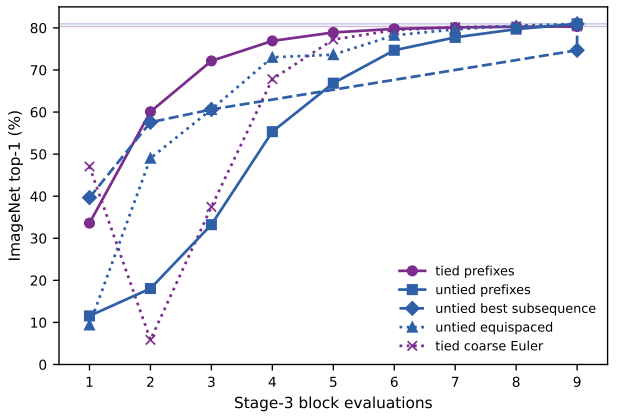


Figure 6. ImageNet top-1 against the number of stage-3 block evaluations (Euler counts 1 per step), without retraining. Purple solid: native tied prefixes (k=1…9). Blue solid: native untied prefixes. Blue dashed: selected untied subsequences from App. E. Blue dotted: equispaced untied subsets $\mathrm{round}(\mathrm{linspace}(0, 8, L))$ with Δt=9/L (Table 14). Purple dotted: coarse Euler of the tied stage at T=9 (App. B). Horizontal lines: full-depth accuracy of each model. The tied stage dominates at few evaluations through its prefixes; the untied stage wins at nine; coarse Euler of the tied stage is not the way to save compute.

Why does one field nearly match nine? We now have a partial answer. The shared field reaches the decoder's operating region earlier (Section 5), and extra depth with the same field recovers none of the 0.62-point gap (Section 4.1). A hypothesis, not a tested claim: what the untied stage buys is the freedom for the field to depend on depth, not more applications of one map. A shared block conditioned on t would test that reading. We do not claim that the untied directions are unnecessary, nor that the blocks compute functions simpler than their parameter counts allow; that remains an untested hypothesis for future work.

Refinement as a probe beyond ConvNeXt. Every architecture converges to a continuous limit at fixed T that

Table 4. Headline measurements on ImageNet-1k. Top-1 and ODE limit from Table 1; straightness (Str.) and consecutive block cosine at $R{=}1$ for the untied families and at $R{=}100$ for the tied stage (Table 3 and Appendix I); parameters from Section 4.1. Tied $R{=}27$: same parameters as Tied, trained at $T{=}27$. Single runs with unmatched recipes. Swin-T and the ResNets are torchvision checkpoints that we did not retrain.

Model	Params	$R{=}1$	ODE	Gap	Str.	$\cos_{k,k+1}$
ConvNeXt-T	28.59M	80.96	80.23	-0.73	0.50	0.16
Weight-tied S3	18.97M	80.34	80.12	-0.22	0.78	0.92
Tied, 27 apps.	18.97M	80.26	–	–	–	–
All-stages tied	8.68M	78.99	–	–	–	–
Swin-T	–	81.08	80.32	-0.76	0.80	0.43
ResNet-50	–	80.34	79.43	-0.91	0.37	-0.11
ResNet-101	–	81.67	81.05	-0.62	0.16	0.01

keeps the function within 0.22 to 0.91 points, although it sits 11° to 21° from the native state (Appendix C); breaking the horizon instead collapses every network. A trained stage is a sequence of one-step solves, each replaceable by a finer integral of the same block, but not a Neural ODE with weights smooth in depth (Appendix F). The probe needs no retraining, the Gram statistics only need block increments and prefix decoding only needs the trained downstream network, so all three apply to any residual stream, including looped transformers (Dehghani et al., 2019; Giannou et al., 2023) and the residual streams of large language models.

Toward mostly autonomous fields. The two regimes suggest a design space between them: a shared field with small per-block corrections, or cycles of a few shared blocks, would interpolate between an autonomous and a switched flow. Related schemes exist for convolutional networks (Savarese & Maire, 2019) and transformers, through cyclic layer sharing (Takase & Kiyono, 2023) and shared layers with low-rank updates (Bae et al., 2025). Designing fields that are mostly autonomous with controlled non-autonomy is future work; the coherence statistics place such models on the ladder from random to autonomous and can test whether coherence and accuracy move together.

Limitations. All results are on ImageNet-1k classification. The tied and untied stages are one pair of ConvNeXt-T runs, without seeds and with different learning-rate schedules; the 27-application tied run is likewise a single run. We describe how the two transports differ and give a partial account of the cost; we do not claim a causal explanation of why the gap is only 0.62 points. The channel basis is not privileged, so per-coordinate readings of the trajectories are partly determined by the Gram matrix and partly by the basis (Appendix H); we base our claims on Gram statistics only. The metrics exclude the massive ConvNeXt channel and the Swin corner tokens, and we have not tested whether the ReLU after the ResNet sum explains its negatively correlated increments (Appendix I). The frozen decoder measures readiness for the trained head, not information content.

7. Conclusion

Refining residual stages at a fixed horizon is a training-free probe: on ConvNeXt, Swin and ResNet the continuous limit costs less than one top-1 point, whereas changing the horizon collapses every network, and the refined path stays faithful to the native Gram geometry. A trained stage is a switched composition of one ODE per block, not one ODE with weights smooth in depth. In ConvNeXt-T the nine blocks write nearly orthogonal, positively correlated directions that no later block reverses, with straightness 0.50 and turns of about 80° at every boundary. Weight tying turns the same stage into one autonomous, anytime flow that a frozen decoder reads much earlier, at a cost of 0.62 points that extra depth does not buy. The open design question is how to keep most of that autonomy while allowing a controlled dependence on depth.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Ainsworth, S. K., Hayase, J., and Srinivasa, S. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations (ICLR)*, 2023.

Alain, G. and Bengio, Y. Understanding intermediate layers using linear classifier probes. In *International Conference on Learning Representations (ICLR), Workshop Track*, 2017. arXiv:1610.01644.

Ansuini, A., Laio, A., Macke, J. H., and Zoccolan, D. Intrinsic dimension of data representations in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Bae, S., Fisch, A., Harutyunyan, H., Ji, Z., Kim, S., and Schuster, T. Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA. In *International Conference on Learning Representations (ICLR)*, 2025.

Bai, S., Kolter, J. Z., and Koltun, V. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

Bansal, A., Schwarzschild, A., Borgnia, E., Emam, Z., Huang, F., Goldblum, M., and Goldstein, T. End-to-

end algorithm synthesis with recurrent networks: Extrapolation without overthinking. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Belrose, N., Furman, Z., Smith, L., Halawi, D., Ostrovsky, I., McKinney, L., Biderman, S., and Steinhardt, J. Eliciting latent predictions from transformers with the tuned lens, 2023. arXiv preprint.

Bengio, Y., Courville, A., and Vincent, P. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35 (8):1798–1828, 2013.

Boulch, A. ShaResNet: reducing residual network parameter number by sharing weights, 2017. URL https://arxiv.org/abs/1702.08782.

Brahma, P. P., Wu, D., and She, Y. Why deep learning works: A manifold disentanglement perspective. *IEEE Transactions on Neural Networks and Learning Systems*, 27(10), 2016.

Chang, B., Meng, L., Haber, E., Tung, F., and Begert, D. Multi-level residual networks from dynamical systems view. In *International Conference on Learning Representations (ICLR)*, 2018.

Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

Cohen, A.-S., Cont, R., Rossier, A., and Xu, R. Scaling properties of deep residual networks. In *International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pp. 2039–2048, 2021.

Cohen, U., Chung, S., Lee, D. D., and Sompolinsky, H. Separability and geometry of object manifolds in deep neural networks. *Nature Communications*, 11:746, 2020.

Darcet, T., Oquab, M., Mairal, J., and Bojanowski, P. Vision transformers need registers. In *International Conference on Learning Representations*, 2024.

Dasgupta, S. and Gupta, A. An elementary proof of a theorem of Johnson and Lindenstrauss. *Random Structures & Algorithms*, 22(1):60–65, 2003.

Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, Ł. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 248–255, 2009.

DiCarlo, J. J. and Cox, D. D. Untangling invariant object recognition. *Trends in Cognitive Sciences*, 11(8):333–341, 2007.

Dupont, E., Doucet, A., and Teh, Y. W. Augmented neural ODEs. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

E, W. A proposal on machine learning via dynamical systems. *Communications in Mathematics and Statistics*, 5 (1):1–11, 2017.

Entezari, R., Sedghi, H., Saukh, O., and Neyshabur, B. The role of permutation invariance in linear mode connectivity of neural networks. In *International Conference on Learning Representations (ICLR)*, 2022.

Fernando, J. and Guitchounts, G. Transformer dynamics: A neuroscientific approach to interpretability of large language models, 2025.

Gan, C., Tu, Y., Chen, X., Chen, T., Li, Y., Harandi, M., and Lin, W. Unleashing diffusion transformers for visual correspondence by modulating massive activations. In *Advances in Neural Information Processing Systems*, volume 38, 2025.

Geiping, J., McLeish, S., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A., and Goldstein, T. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Giannou, A., Rajput, S., yong Sohn, J., Lee, K., Lee, J. D., and Papailiopoulos, D. Looped transformers as programmable computers. In *International Conference on Machine Learning (ICML)*, 2023.

Glorot, X. and Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256, 2010.

Greff, K., Srivastava, R. K., and Schmidhuber, J. Highway and residual networks learn unrolled iterative estimation. In *International Conference on Learning Representations (ICLR)*, 2017.

Gromov, A., Tirumala, K., Shapourian, H., Glorioso, P., and Roberts, D. A. The unreasonable ineffectiveness of the deeper layers. In *International Conference on Learning Representations (ICLR)*, 2025.

Haber, E. and Ruthotto, L. Stable architectures for deep neural networks. *Inverse Problems*, 34(1):014004, 2017.

Hairer, E., Nørsett, S. P., and Wanner, G. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, 2 edition, 1993.

Halko, N., Martinsson, P.-G., and Tropp, J. A. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

Hauser, M. and Ray, A. Principles of Riemannian geometry in neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

Hénaff, O. J., Goris, R. L. T., and Simoncelli, E. P. Perceptual straightening of natural videos. *Nature Neuroscience*, 22(6):984–991, 2019.

Hosseini, E. A. and Fedorenko, E. Large language models implicitly learn to straighten neural sentence trajectories to construct a predictive representation of natural language. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Huang, G., Sun, Y., Liu, Z., Sedra, D., and Weinberger, K. Q. Deep networks with stochastic depth. In *European Conference on Computer Vision (ECCV)*, 2016.

Huang, G., Chen, D., Li, T., Wu, F., van der Maaten, L., and Weinberger, K. Q. Multi-scale dense networks for resource efficient image classification. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=HkfUCi1C-.

Ioffe, S. and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015.

Jastrzębski, S., Arpit, D., Ballas, N., Verma, V., Che, T., and Bengio, Y. Residual connections encourage iterative inference. In *International Conference on Learning Representations (ICLR)*, 2018.

Johnson, W. B. and Lindenstrauss, J. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.

Kaya, Y., Hong, S., and Dumitras, T. Shallow-deep networks: Understanding and mitigating network overthinking. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 3301–3310. PMLR, 2019. URL https://proceedings.mlr.press/v97/kaya19a.html.

Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 3519–3529. PMLR, 2019. URL https://proceedings.mlr.press/v97/kornblith19a.html.

Lad, V., Lee, J. H., Gurnee, W., and Tegmark, M. The remarkable robustness of LLMs: Stages of inference? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2406.19384.

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., and Soricut, R. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations (ICLR)*, 2020.

Liao, Q. and Poggio, T. Bridging the gaps between residual learning, recurrent neural networks and visual cortex, 2016.

Liberzon, D. *Switching in Systems and Control*. Birkhäuser, Boston, 2003.

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. Swin Transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Lu, Y., Zhong, A., Li, Q., and Dong, B. Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations. In *International Conference on Machine Learning (ICML)*, 2018.

Marion, P., Wu, Y.-H., Sander, M. E., and Biau, G. Implicit regularization of deep residual networks towards neural ODEs. In *International Conference on Learning Representations (ICLR)*, 2024.

Naitzat, G., Zhitnikov, A., and Lim, L.-H. Topology of deep neural networks. *Journal of Machine Learning Research*, 21(184):1–40, 2020.

Pandey, V., Singh, G., and Mahdid, Y. Trajectory geometry of transformer representations across layers, 2026. URL https://arxiv.org/abs/2606.09287.

Queiruga, A. F., Erichson, N. B., Taylor, D., and Mahoney, M. W. Continuous-in-depth neural networks, 2020. URL https://arxiv.org/abs/2008.02389.

Sander, M. E., Ablin, P., and Peyré, G. Do residual neural networks discretize neural ordinary differential equations? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Savarese, P. and Maire, M. Learning implicitly recurrent CNNs through parameter sharing. In *International Conference on Learning Representations (ICLR)*, 2019.

Strogatz, S. H. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. Westview Press, 2 edition, 2015.

Sun, M., Chen, X., Kolter, J. Z., and Liu, Z. Massive activations in large language models. In *Conference on Language Modeling (COLM)*, 2024.

Sun, Q., Pickett, M., Nain, A. K., and Jones, L. Transformer layers as painters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 25219–25227, 2025.

Sun, S., Canziani, A., LeCun, Y., and Zhu, J. Anatomy of massive activations and attention sinks. In Zhang, T., Dudik, M., Jaggi, M., Agarwal, A., Li, S., Schuurmans, D., Zhu, J., Berkenkamp, F., Dong, H., and Bietti, A. (eds.), *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pp. 116443–116463. PMLR, 06–11 Jul 2026. URL https://proceedings.mlr.press/v306/sun26d.html.

Takase, S. and Kiyono, S. Lessons on parameter sharing across layers in transformers. In *Proceedings of the Fourth Workshop on Simple and Efficient Natural Language Processing (SustaiNLP)*, pp. 78–90, 2023. doi: 10.18653/v1/2023.sustainlp-1.5.

Teerapittayanon, S., McDanel, B., and Kung, H. T. BranchyNet: Fast inference via early exiting from deep neural networks. In *Proceedings of the 23rd International Conference on Pattern Recognition (ICPR)*, pp. 2464–2469. IEEE, 2016. doi: 10.1109/ICPR.2016.7900006.

Valeriani, L., Doimo, D., Cuturello, F., Laio, A., Ansuini, A., and Cazzaniga, A. The geometry of hidden representations of large transformer models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Veit, A., Wilber, M., and Belongie, S. Residual networks behave like ensembles of relatively shallow networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.

A. Training recipe

We train on ImageNet-1k (Deng et al., 2009) at 224×224. Table 5 lists the full hyper-parameter table, next to the official ConvNeXt-T recipe (Liu et al., 2022). The models we train from scratch share the same augmentation, regularisation and optimiser; they differ in peak learning rate, warmup length, minimum learning rate, stochastic depth, stage-3 depth and parallelism, as marked in the table. The official ConvNeXt-T uses batch size 4096 and base learning rate 4×10^{-3}. Linear scaling to our batch of 1024 gives 1×10^{-3}, which is the peak we use for the weight-tied, D=27 and all-stages runs. The untied ConvNeXt-T and its drop-path-free sibling use half of that, 5×10^{-4}, with the official 20-epoch warmup. The untied runs use one process with `DataParallel` over a batch of 1024; the tied, D=27 and all-stages runs use four distributed processes with 256 images each. Mixup and CutMix are drawn per process batch, so the mixing statistics are not identical between the two set-ups. No run fixes a random seed, so the accuracy gaps of Section 4.1 come from single runs under recipes that are close but not matched.

Weight decay is applied only to convolution and linear weights (tensors of rank ≥ 2). LayerNorm gain and bias, convolution/linear bias and LayerScale γ are decay-free, following the ConvNeXt / DeiT convention. Convolution and linear weights are initialised with truncated normal $\mathcal{N}(0, 0.02^2)$; LayerScale uses γ=10^{-6}. There is no layer-wise learning-rate decay, no gradient clipping, no mixed precision and no exponential moving average of the weights. The learning-rate schedule is a linear warmup (starting at 10^{-6}) followed by cosine decay to the listed minimum, advanced once per optimiser step.

Mixup and CutMix are applied to every training batch (switch probability 0.5), and the loss is soft-target cross-entropy on the mixed labels. Per-image augmentation is the timm ConvNeXt stack: random resized crop, horizontal flip, RandAugment `rand-m9-mstd0.5`, color jitter 0.4 and random erasing (probability 0.25, pixel mode). Validation uses the standard centre crop. Stochastic depth, when enabled, is ramped linearly from 0 to the listed peak across the 18 residual blocks of the untied model. In the weight-tied model it is applied per application of the shared block: the k-th application in stage 3 uses the rate of the k-th untied stage-3 block, $\mathrm{linspace}(0, 0.1, 18)[6{+}k]$, i.e. 0.035 to 0.082, so both models drop the same expected fraction of stage-3 updates. For the D=27 run the same per-application schedule is extended over the 27 applications (`scripts/train_shared_convnext_D27.py`).

We do not report decoder fine-tuning after refinement. An earlier result that raised the tied 128-step top-1 from 80.14% to 80.33% used a script that is no longer in the repository and may have left stochastic depth active on frozen stages; the corrected protocol (`scripts/finetune_s4_head_protocol.py`) has no completed runs in `outputs/`, so we omit that claim.

B. Full Midpoint and RK4 refinement tables

We use the same R (repetitions) as in the main text: Δt=$1/R$ so that T=n is conserved (n=9 for ConvNeXt stage 3). Tables 6–9 report the piecewise-constant schedule for Euler (RK1), Midpoint (RK2) and RK4 on each pretrained model. Table 10 gives the same solvers for the weight-tied ConvNeXt-T at fixed horizon T=9 (9R applications of the shared block). In every case the three integrators converge to the same continuous limit once the step is fine enough; at R=1, RK4 already sits on that limit on every model, and Midpoint does on ConvNeXt but not on the ResNets. The ConvNeXt-T column is the evaluation of the final checkpoint; repeated evaluations of the same checkpoint differ by up to two images (0.004 points) because of nondeterministic GPU kernels.

Large steps and the trained horizon. Figure 7 plots the fractional rows of Table 10. Those coarse steps evidence the trained horizon; they are not a general compute advantage of the tied stage. With three Euler steps of size 3 the tied model reaches only 37.5%, below the 60.6% of the untied blocks $0, 4, 8$ with the same Δt=3 (Appendix E), so saving evaluations by enlarging Δt at fixed T favours neither model in a simple way. Euler is not monotone in the number of steps: one step of Δt=9 keeps 47.1%, two steps of 4.5 fall to 5.9% and three steps of 3 recover 37.5%. The two-step path overshoots. On 500 images, the stage-3 exit norm without the massive channel is 1370 after two steps, three times the native 458, against 452 after one step and 683 after three; the single step lands at a normal norm in a different direction (cosine 0.41 to the native exit), which the decoder still partly classifies. Varying the number of unit steps at Δt=1, or Δt at nine steps, peaks at T=9 in both cases (Figure 8). Script for the overshoot measurements: `scripts/undo_geometry_audit.py`.

C. Solver agreement at the stage-3 endpoint

Section 3.2 asks whether a fine Euler step recovers the native R=1 state. A related question is whether two fine discretisations of the *same* residual field agree with each other, that is, whether a continuous limit exists at all. Table 11 compares stage-3 endpoints x_T on the same N=32 ImageNet validation images (one per random class, seed 0), after zeroing the massive channel used in the main-text metrics. Relative distance is $\|x_a - x_b\|/\|x_b\|$; cosine is $\cos(x_a, x_b)$.

Euler and RK4 (and Euler R=100 vs. R=200) collapse

Table 5. ImageNet-1k training settings. Official ConvNeXt-T from Liu et al. (2022); the remaining columns are the models we train. CNX-T: untied baseline. Tied: stage 3 shares one block (9 applications, T=9, Δt=1). D=27: same shared block applied 27 times, trained from scratch at T=27, Δt=1 (same parameter count as Tied). All: one shared block per stage. dp0: untied ConvNeXt-T with stochastic depth off.

	Official	CNX-T	Tied	D=27	All	dp0
training epochs	300	300	300	300	300	300
warmup epochs	20	20	10	10	10	20
optimizer	AdamW	AdamW	AdamW	AdamW	AdamW	AdamW
base learning rate	4×10^{-3}	5×10^{-4}	1×10^{-3}	1×10^{-3}	1×10^{-3}	5×10^{-4}
min. learning rate	–	10^{-6}	10^{-7}	10^{-7}	10^{-7}	10^{-6}
weight decay	0.05	0.05	0.05	0.05	0.05	0.05
optimizer momentum	β=(0.9, 0.999)	β=(0.9, 0.999)	β=(0.9, 0.999)	β=(0.9, 0.999)	β=(0.9, 0.999)	β=(0.9, 0.999)
batch size	4096	1024	1024	1024	1024	1024
processes × batch	–	1×1024	4×256	4×256	4×256	1×1024
stage-3 applications	9	9	9	27	9	9
trained horizon T	9	9	9	27	9	9
random seed	–	none	none	none	none	none
learning rate schedule	cosine	cosine	cosine	cosine	cosine	cosine
warmup schedule	linear	linear	linear	linear	linear	linear
layer-wise lr decay	None	None	None	None	None	None
RandAugment	(9, 0.5)	(9, 0.5)	(9, 0.5)	(9, 0.5)	(9, 0.5)	(9, 0.5)
Mixup α	0.8	0.8	0.8	0.8	0.8	0.8
CutMix α	1.0	1.0	1.0	1.0	1.0	1.0
random erasing	0.25	0.25	0.25	0.25	0.25	0.25
color jitter	0.4	0.4	0.4	0.4	0.4	0.4
label smoothing	0.1	0.1	0.1	0.1	0.1	0.1
stochastic depth	0.1	0.1	0.1	0.1	0.1	0
LayerScale init	10^{-6}	10^{-6}	10^{-6}	10^{-6}	10^{-6}	10^{-6}
gradient clip	None	None	None	None	None	None
exp. moving average	None	None	None	None	None	None
mixed precision	–	off	off	off	off	off

Table 6. ConvNeXt-T, piecewise-constant schedule, top-1 (%).

R	Euler	Mid	RK4
1	80.96	80.23	80.23
2	80.78	80.23	80.23
4	80.52	80.23	80.23
10	80.36	80.23	80.23
32	80.29	80.23	80.23
128	80.24	80.23	80.23

Table 7. Swin-T, piecewise-constant schedule, top-1 (%).

R	Euler	Mid	RK4
1	81.08	80.29	80.33
2	80.90	80.34	80.31
4	80.65	80.33	80.32
10	80.47	80.32	80.31
32	80.38	80.32	80.32
128	80.34	80.31	80.32

Table 8. ResNet-50, piecewise-constant schedule, top-1 (%).

R	Euler	Mid	RK4
1	80.34	78.94	79.42
2	80.06	79.41	79.43
4	79.84	79.44	79.43
10	79.61	79.44	79.43
32	79.50	79.43	79.44
128	79.45	79.43	79.43

Table 9. ResNet-101, piecewise-constant schedule, top-1 (%).

R	Euler	Mid	RK4
1	81.67	80.95	81.06
2	81.58	81.07	81.04
4	81.38	81.06	81.05
10	81.22	81.05	81.05
32	81.12	81.05	81.04
128	81.06	81.04	81.05

onto the same x_T, while both remain far from the trained one-step path. RK4 reaches that limit already with the nine unit steps of the native schedule; the native Euler path does not. Midpoint at R=1 matches the limit in top-1 (80.23%, Table 6) while its state is still 12% away, a reminder that accuracy is a coarse probe of x_T.

Table 10. Shared ConvNeXt-T, fixed T=9 (Δt=1/R; 9R shared steps), top-1 (%).

R	Steps	Δt	Euler	Mid	RK4
1/9	1	9	47.06	0.61	2.70
2/9	2	4.5	5.85	10.42	47.98
3/9	3	3	37.45	54.47	71.37
4/9	4	2.25	67.83	73.57	78.90
5/9	5	1.8	77.23	78.70	79.97
6/9	6	1.5	79.48	79.85	80.10
7/9	7	1.29	80.19	80.12	80.11
8/9	8	1.13	80.38	80.21	80.11
1	9	1	80.34	80.19	80.11
4/3	12	0.75	80.35	80.14	80.12
2	18	0.5	80.27	80.12	80.12
32/9	32	0.28	80.22	80.11	80.12
128/9	128	0.07	80.14	80.12	80.12
1024/9	1024	0.009	80.12	80.12	80.12
10000/9	10000	0.0009	80.12	–	–

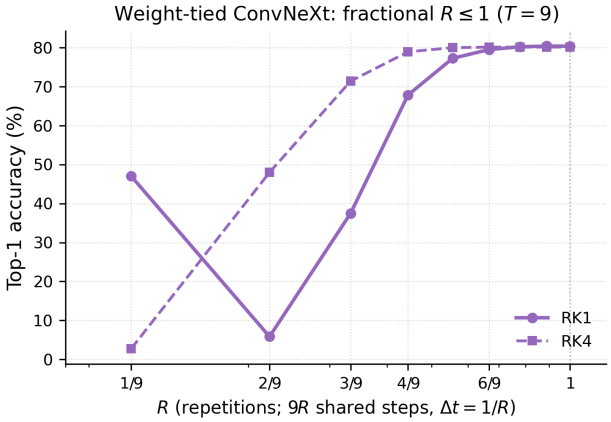


Figure 7. Weight-tied ConvNeXt-T at fixed T=9 with $9R \leq 9$ steps (Euler solid, RK4 dashed).

Accumulated and bounded refinement error. Figure 9 follows the gap between the refined (R=100) and the native path along the whole stage, on the 500 images of the main text. After the first native step both models have cosine 0.99 to the native path. In the untied ConvNeXt-T the block-end cosine keeps falling, to 0.93 at t=9 (21.4°), and the distance grows from 5.6 to 76.4 while the norms differ by 3% (203.5 against 210.3): each block receives an input that has already drifted, and the gap is mostly a rotation. In the weight-tied model the cosine settles at about 0.98 after two or three steps and stays there (11.1° at t=9); its refined path ends 7% shorter (425.5 against 458.4), so part of its smaller relative gap (0.20 against 0.36) is radial.

D. Random-initialisation control for the continuous limit

Appendix C shows that the trained stage 3 has a continuous limit. That alone is weak evidence. Every residual

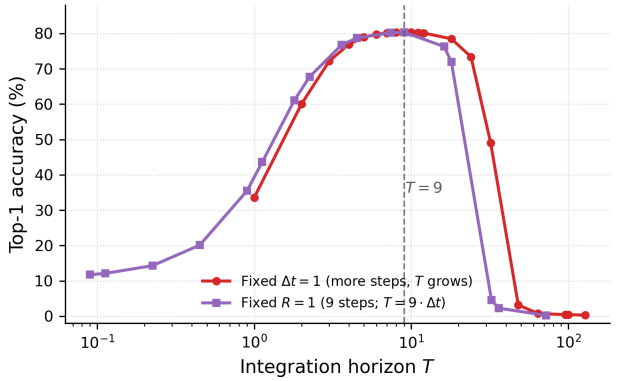


Figure 8. Weight-tied ConvNeXt-T: top-1 against horizon T. Red: fixed Δt=1 and more steps. Purple: nine steps with varying Δt (T=9 Δt). Both peak at the trained horizon T=9.

Table 11. Pairwise stage-3 endpoint agreement (mean over N=32; ignore top-1 channel). Horizon T=9 in every row.

Pair (a vs. b)	rel. ℓ_2	cos	angle
ConvNeXt-T			
Euler R=100 vs. RK4 R=100	0.004	1.000	0.25°
Euler R=100 vs. Euler R=200	0.002	1.000	0.12°
Euler R=100 vs. Euler R=1	0.36	0.93	21°
Mid R=1 vs. Euler R=100	0.12	0.993	6.6°
RK4 R=1 vs. Euler R=100	0.010	0.9999	0.5°
Weight-tied ConvNeXt-T (9R shared steps)			
Euler R=100 vs. RK4 R=100	0.002	1.000	0.1°
Euler R=100 vs. Euler R=200	0.001	1.000	0.05°
Euler R=100 vs. Euler R=1	0.20	0.98	11°
RK4 R=1 vs. RK4 R=100	0.003	1.000	0.16°

field we study (convolutions, linear layers, normalisation, GELU/ReLU, softmax attention) is locally Lipschitz with at most linear growth (attention outputs are convex combinations of the values), so the ODE $\dot{x} = f_i(x)$ has a unique solution on $[0, T]$ and Euler converges to it at first order, whatever the weights (Hairer et al., 1993). We check this empirically on all five architectures and use random fields of controlled strength as a reference scale for the trained ones.

Setup. Every convolution and linear layer is drawn with Xavier-normal initialisation (Glorot & Bengio, 2010) (gain 1, zero bias, seed 0); normalisation layers keep their default parameters, and ConvNeXt LayerScale is set to γ=1 (its default 10^{-6} makes $f \approx 0$). The stage-3 blocks are then re-drawn with Xavier gain $g \in \{0.25, 0.5, 1, 2, 4, 8\}$, so the stage-3 input is identical across gains and only the field strength changes. For ResNet the field is $f_i(x) = \mathrm{block}_i(x) - x$, which includes the ReLU applied after the addition, and the first stage-3 block, whose projection shortcut changes the resolution and width, runs once without refinement, so n=5 for ResNet-50 and n=22 for ResNet-

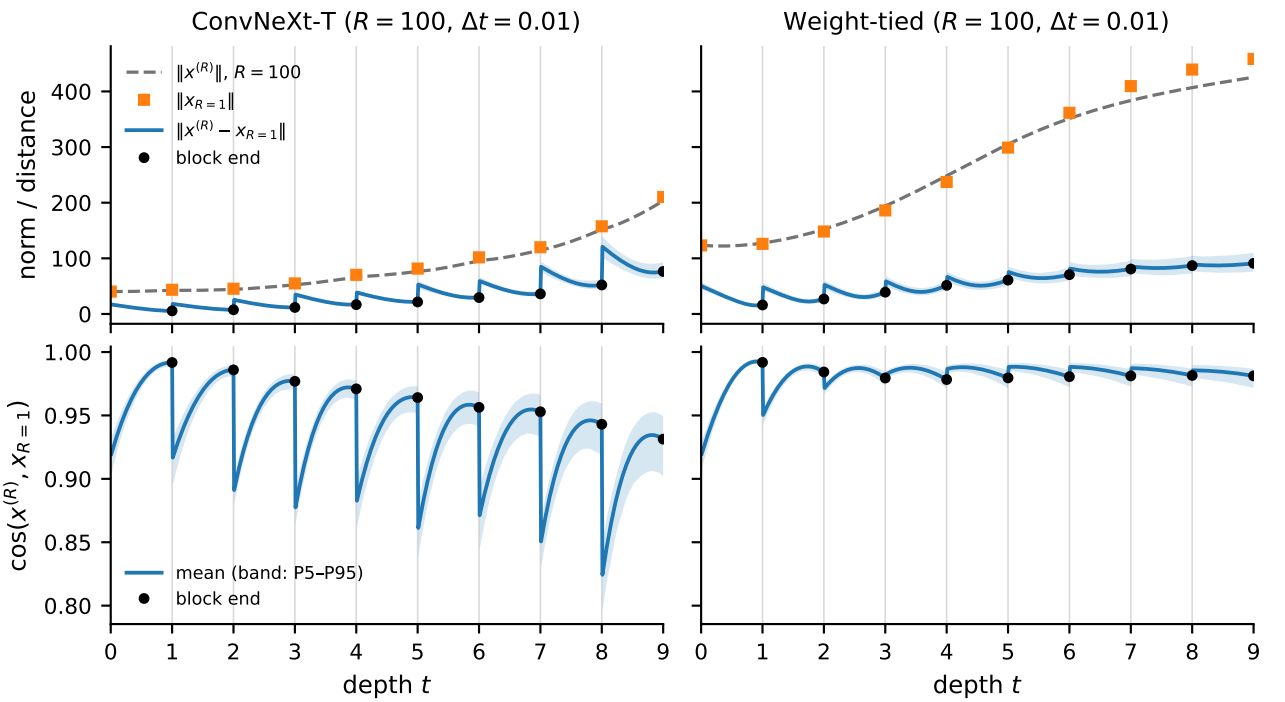


Figure 9. Refined (R=100) against native (R=1) stage-3 path for the untied (left) and weight-tied (right) ConvNeXt-T. **Top:** $\|x^{(R)} - x_{R=1}\|$ (solid) with both state norms for scale (dashed: refined; squares: native). **Bottom:** $\cos(x^{(R)}, x_{R=1})$. Mean and P5–P95 over 500 images; massive channel removed; dots mark the end of each native step.

101 (the same convention holds in every experiment); for Swin one block contains both residual sub-layers (attention and MLP), as in the main text. For each network we integrate stage 3 to $T{=}n$ and compare the endpoint x_T of Euler at $R \in \{1, 10, 100, 1000\}$ and of RK4 at $R{=}1$ against RK4 at $R{=}100$, on the $N{=}32$ images of Appendix C. The field strength is summarised by the mean relative step of the native path,

$$\rho \;=\; \frac{1}{n}\sum_{i=0}^{n-1}\frac{\|f_i(x_i)\|}{\|x_i\|}, \qquad x_{i+1} = x_i + f_i(x_i), \tag{6}$$

i.e. how far a single block moves the state relative to its size. Because ρ grows with g at a different rate in each architecture (about g^3 for ResNet, whose bottleneck has three unnormalised convolutions), we compare trained and random networks at matched ρ: for each architecture we add one gain $g^\star$ chosen so that ρ matches the trained network. Trained ConvNeXt rows zero the massive channel (Section 4.2); ResNet and Swin have no channel of comparable dominance at stage 3 (zeroing the largest one changes every entry by less than 0.6°), so all rows except those use all channels.

Results. Table 12 supports two observations.

(i) The limit exists for every architecture. For every row, trained or random, the Euler error falls by roughly 10× per decade of R, the first-order rate (trained Swin-T is slower between $R{=}10$ and $R{=}100$), and Euler at $R{=}1000$ is within 0.1° of the reference. The same holds at the larger gains not shown, with two numerical exceptions that confirm the mechanism rather than contradict it. ResNet at $g \geq 4$ overflows in float32: without a normalisation inside the block, $\|f_i(x)\|$ grows with $\|x\|$, so the state grows exponentially ($\rho \approx 18$ per block at $g{=}4$). Swin at $g{=}8$ is still 15.7° away at $R{=}1000$: the attention logits are so large that the softmax is nearly a hard selection, the field becomes very stiff and a finer step is needed. Convergence of the refinement to a single $\hat{x}_T$ is therefore a property of the residual parameterisation, not something learned.

(ii) The native gap is of the order set by ρ, and training moves it in an architecture-dependent direction. Within each architecture the distance from the single-step path ($R{=}1$) to the limit grows monotonically with ρ, and with $\rho \approx 0.3$–0.9 every trained network lies 10–21° away: one Euler step of size 1 is far from tracking the flow. At matched ρ, however, the trained and random networks differ, and not in a consistent direction. Trained Swin-T and ConvNeXt-T are *closer* to their limit than a random field of the same strength (19.5° vs. 47.6° and 21.1° vs. 30.2°), the weight-tied model is level with it (10.9° vs. 11.9°), and both ResNets are *farther* (17.0° vs. 12.0° and 15.3° vs. 8.9°).

Table 12. Angle between x_T and the RK4 R=100 reference (T=n, mean over N=32). g: Xavier gain of the stage-3 blocks; ρ: mean relative step (6); *: gain matched to the trained ρ. Rows sorted by ρ.

Init	ρ	Euler			RK4
		R=1	R=10	R=100	R=1
ConvNeXt-T (nine blocks)					
g=0.5	0.13	1.5°	0.15°	0.02°	0.00°
g=1	0.41	15.8°	1.6°	0.16°	0.07°
Trained	**0.54**	**21.1°**	**2.4°**	**0.25°**	**0.53°**
$g^\star$=1.35	0.56	30.2°	3.4°	0.34°	0.92°
g=2	0.77	47.0°	6.7°	0.69°	8.4°
Weight-tied ConvNeXt-T (one block)					
g=0.5	0.12	2.7°	0.27°	0.03°	0.00°
$g^\star$=1	0.30	11.9°	1.3°	0.13°	0.05°
Trained	**0.31**	**10.9°**	**1.0°**	**0.10°**	**0.16°**
g=2	0.63	27.1°	4.3°	0.46°	5.9°
ResNet-50 (n=5)					
g=0.5	0.04	0.4°	0.03°	0.00°	0.01°
g=1	0.34	7.0°	0.66°	0.07°	0.24°
$g^\star$=1.15	0.51	12.0°	1.2°	0.12°	0.48°
Trained	**0.52**	**17.0°**	**1.4°**	**0.15°**	**0.65°**
g=2	2.29	40.6°	7.8°	0.83°	6.6°
ResNet-101 (n=22)					
g=0.5	0.05	0.7°	0.05°	0.01°	0.02°
Trained	**0.30**	**15.3°**	**1.4°**	**0.15°**	**0.74°**
$g^\star$=0.95	0.30	8.9°	0.83°	0.08°	0.29°
g=1	0.35	10.3°	0.97°	0.10°	0.36°
g=2	2.33	39.6°	5.9°	0.63°	6.4°
Swin-T (n=6)					
g=0.5	0.15	1.7°	0.17°	0.02°	0.00°
g=1	0.48	17.2°	1.8°	0.18°	0.10°
Trained	**0.87**	**19.5°**	**4.3°**	**1.02°**	**2.7°**
$g^\star$=1.8	0.89	47.6°	7.1°	0.74°	6.6°
g=2	0.98	52.7°	8.7°	0.91°	10.5°

RK4 at Δt=1 follows the same split. Training thus changes how well a single step approximates the flow, but there is no universal tendency for single-use blocks to be better (or worse) one-step solvers than random ones. Script: `scripts/random_init_convergence.py`.

E. Block selection, reordering and duplication

Section 4.1 trains a stage to use a single function. Here we ask the opposite question on the trained untied ConvNeXt-T, without any retraining: can a subset of its stage-3 blocks, reordered or reused, replace the nine blocks? Each configuration is a sequence of block indices applied with one Euler step Δt, so the horizon is T=$L\,\Delta t$ for a sequence of length L (Table 13; full ImageNet validation). The same evaluation run gives 80.79, 80.53 and 80.36% for the refined schedules R=2, 4, 10, matching Table 1 within 0.01 points, so it uses the same checkpoint. Script: `scripts/convnext_interpolation.py`.

Table 13. Untied ConvNeXt-T stage 3 replaced by sequences of its own blocks, top-1 (%). T: horizon $L\,\Delta t$. Native nine blocks: 80.96.

Blocks	Δt	T	top-1
best single block (5)	9	9	39.67
best single block (5)	1	1	43.12
0, 8 / 8, 0	4.5	9	49.03 / 15.31
1, 7 / 7, 1	4.5	9	23.14 / 15.12
2, 6 / 6, 2	4.5	9	13.81 / 3.97
3, 5 / 5, 3	4.5	9	57.57 / 35.18
4, 4	4.5	9	30.73
0, 4, 8	4.5	13.5	38.16
0, 4, 8	3	9	60.62
0, 4, 8	2.25	6.75	66.57
$(0, 4, 8) \times 3$	1	9	52.42
0, 0, 2, 2, 4, 4, 6, 6, 8	1	9	74.69
0, 0, 0, 3, 3, 3, 6, 6, 6	1	9	55.13
0, 0, 0, 0, 4, 8, 8, 8, 8	1	9	54.60

No single block suffices: the best one, block 5, reaches 39.7% when integrated over the whole horizon in one step and 43.1% with a unit step, and block 6 peaks at 29.2% near Δt=1 over a sweep from 0.1 to 20. With two blocks the original order wins in all four pairs we tested in both orders, by 8.0 to 33.7 points. Using every other block twice keeps 74.7%, the best substitute we found, whereas cycling blocks $0, 4, 8$ three times gives 52.4%. The blocks are therefore not interchangeable copies of one operation, consistent with the nearly orthogonal increments of Section 4.3 and with the finding that repeating one middle layer is the most harmful intervention in pretrained transformers (Sun et al., 2025).

Equispaced subsets at fixed horizon. Table 14 reports the systematic family used in Figure 6: for each length L=1…9, the indices $\mathrm{round}(\mathrm{linspace}(0, 8, L))$ (duplicates removed) with step Δt=$9/L$, so T=9. L=2 and L=3 reproduce the App. E rows for $\{0, 8\}$ and $\{0, 4, 8\}$; L=9 recovers the native stage. Script: `paper/scripts/plot_block_evals.py` (calls the same validation path as `scripts/convnext_interpolation.py`); results in `outputs/block_evals_figure/equispaced_results.csv`.

F. Bilinear weight interpolation

Continuous-in-depth work treats the weights as a function of depth $\theta(t)$ expanded in a basis (Queiruga et al., 2020). The piecewise-constant basis recovers a ResNet, and multi-level training refines depth by copying adjacent blocks (Chang et al., 2018); smoother bases are a natural alternative. We test the simplest one, linear interpolation between consecutive blocks,

$$\theta(t) \;=\; (1-\alpha)\,\theta_i + \alpha\,\theta_{i+1}, \qquad \alpha = t - i. \tag{7}$$

Table 14. Equispaced untied stage-3 subsets at T=9, ImageNet top-1 (%). Indices: $\mathrm{round}(\mathrm{linspace}(0, 8, L))$, unique, order preserved.

L	Blocks	Δt	top-1
1	0	9	9.40
2	0, 8	4.5	49.03
3	0, 4, 8	3	60.62
4	0, 3, 5, 8	2.25	73.01
5	0, 2, 4, 6, 8	1.8	73.64
6	0, 2, 3, 5, 6, 8	1.5	78.25
7	0, 1, 3, 4, 5, 7, 8	1.286	79.64
8	0, 1, 2, 3, 5, 6, 7, 8	1.125	80.53
9	0, 1, 2, 3, 4, 5, 6, 7, 8	1	80.96

Replacing the piecewise-constant schedule $\theta_t = \theta_{\lfloor t \rfloor}$ by (7) hurts every model we tested: at R=128 the drop against the piecewise-constant schedule is between 6.6 and 23.3 points (Table 15). Nothing aligns the hidden units of consecutive blocks; as for independently trained networks, averaging weights that are not permutation-aligned need not give a functional intermediate (Entezari et al., 2022; Ainsworth et al., 2023).

Figure 10 shows the ConvNeXt residual-field norm under the piecewise-constant and bilinear schedules at R=100. The piecewise-constant schedule keeps the staircase of Figure 2 (right); bilinear adds high-frequency oscillation between block boundaries while preserving the same global growth of $\|f\|$. Bands are P5–P95 across 500 validation images (same suite as Figure 2).

G. Massive activations in stage 3

Section 4.2 summarises the outliers found in stage 3 of ConvNeXt and Swin. Here we detail how they are identified, how we ablate the ConvNeXt channel and what their kinematics look like. Throughout, we use "massive activation" loosely for channels or tokens that exceed all others by more than an order of magnitude. On the shared and untied ConvNeXt the stage-3 exit also meets the numerical criterion of Sun et al. (2024) (magnitude above 100 and about 10^3 times the median), although here the dominant channel is large at every position rather than at a few tokens, a pattern closer to the uniformly large feature dimension that Sun et al. (2024) report for MAE ViTs and do not count as massive: at every spatial position $\max_c |x_c| > 100$, and the median over positions of $\max_c |x_c| / \mathrm{median}_c |x_c|$ is about $1.1{\times}10^3$ (shared, N=500) and $1.3{\times}10^3$ (untied, N=500). In Swin the corner tokens fail that channel-wise test (ratios of about 16), so the label there refers only to their outlying L2 norms.

Detection. On the shared ConvNeXt we integrate the native stage-3 schedule (9 applications of the shared block, Δt=1, Euler) on a single ImageNet validation image and rank the 384 channels by the Frobenius norm $\|x_c\|_F$ of the stage-3 state just before LayerNorm and downsampling. Channel 236 dominates by more than an order of magnitude ($\|x_{236}\|_F \approx 3.6{\times}10^3$ vs. $\approx 1.4{\times}10^2$ for the runner-up; Figure 11). The same index remains the top channel under the refined schedule (R=10, 90 applications, Δt=0.1), so the outlier is not an artefact of the unit-step discretisation. An analogous ranking on the untied ConvNeXt-T likewise isolates one channel of comparable relative size, channel 195; ResNet stage 3 has no channel of this dominance.

Table 15. Bilinear weight interpolation, Euler, top-1 (%).

R	CNX-T	CNX-T†	Swin-T	R-50	R-101
1	80.96	79.27	81.08	80.34	81.67
2	78.94	72.43	78.93	74.95	72.55
4	76.16	63.13	75.42	70.60	62.42
10	74.42	58.75	73.28	68.58	58.73
128	73.66	57.44	72.14	67.77	57.73
Δ at R=128	−6.6	−19.6	−8.2	−11.7	−23.3

†ConvNeXt-T trained without drop path.

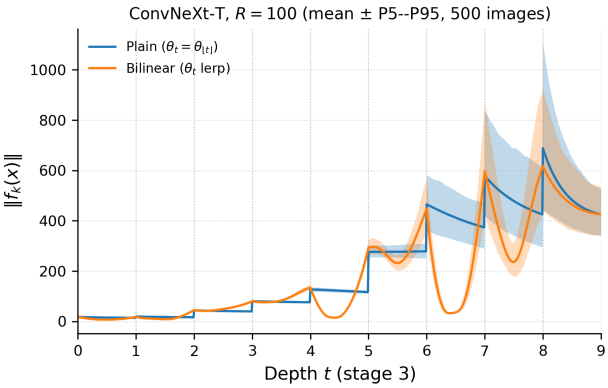


Figure 10. ConvNeXt-T residual norm along stage 3 at R=100: piecewise-constant against bilinear weights (mean and P5–P95 over 500 images).

Zeroing protocol. To test functional importance we re-run the full ImageNet validation pass while forcing the selected channel to zero after *every* residual step of stage 3 (stem and stages 1–2, 4 are untouched). We compare three conditions under both the native (R=1) and refined (R=10) shared schedules, always at the trained horizon T=9: (i) baseline, no channel forced; (ii) outlier, channel 236 zeroed; (iii) random controls, each of 20 channels drawn uniformly from $\{0, \ldots, 383\} \setminus \{236\}$ (seed 0) zeroed independently. Table 16 reports top-1 and cross-entropy; Figure 12 shows the corresponding Δloss histogram. Zeroing the outlier costs about 12.5 pp of top-1 and raises the loss by ≈ 0.69, while every random channel stays within 0.3 pp and $\Delta\mathrm{loss} \leq 0.02$. The drop is essentially unchanged from R=1 to R=10, so the channel remains critical under refinement.

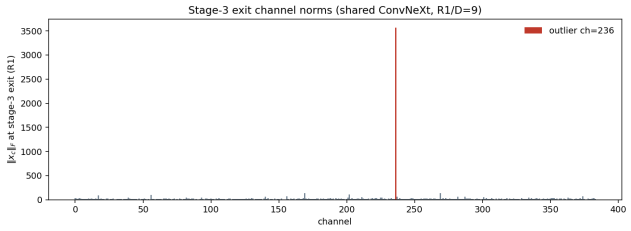


Figure 11. Stage-3 exit channel norms on the shared ConvNeXt (R=1, 9 applications). Channel 236 (red) is the massive activation used in the ablations below.

Table 16. Shared ConvNeXt-T ImageNet validation when zeroing one stage-3 channel after every residual step. Random: mean and range over 20 channels.

R	condition	top-1 (%)	loss	Δloss
1	baseline	80.34	0.896	–
1	zero ch 236	67.81	1.595	+0.699
1	zero random (n=20)	80.25 [80.05, 80.38]	–	$\leq$ 0.018
10	baseline	80.14	0.904	–
10	zero ch 236	67.59	1.590	+0.686
10	zero random (n=20)	80.11 [79.85, 80.18]	–	$\leq$ 0.019

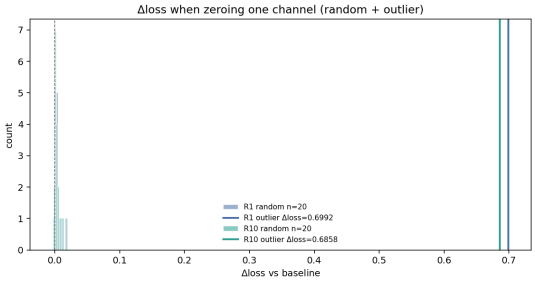


Figure 12. Δloss vs. baseline when zeroing one stage-3 channel on the shared ConvNeXt. Random channels (n=20 per R) cluster near zero; the outlier (vertical lines) sits near $\Delta\text{loss} \approx 0.7$ for both R=1 and R=10.

These numbers show that the trained network relies on this channel, but not that a massive channel is necessary: zeroing it also removes its contribution to the LayerNorm statistics at every spatial position and therefore rescales all other channels.

Phase portraits. Figure 13 plots, for every stage-3 channel, the trajectory of spatial means $(\text{mean}_{H,W}(x_d), \text{mean}_{H,W}(h_d))$ over T=9 at Δt=0.01 (N=500 validation images). In the shared model the outlier traces a smooth arc: the residual mean $\text{mean}(h)$ rises then falls while the state mean $\text{mean}(x)$ keeps growing, matching the early acceleration and late velocity drop reported in the main text. In the untied ConvNeXt-T the dominant channel advances in a block-wise sawtooth and moves away from the origin, its magnitude growing with negative sign: its velocity decays within most blocks and jumps at every block boundary. In both models the spatial means of the remaining 383 channels stay close to the origin. This dominance is why we exclude the outlier channel when computing the trajectory-geometry metrics of the main text; the forward pass itself is left unchanged.

Swin. In Swin-T the outliers are concentrated in tokens rather than channels, so means hide them: averaging over channels cancels coordinates of opposite sign, and averaging over positions dilutes four tokens among 196. We therefore plot, for each stage-3 position, the norm over channels of the state $\|x_d\|_C$ against that of the residual $\|h_d\|_C$ (Figure 14, left). The four corner tokens end the stage with norms of about 500 to 600, whereas the median norm over the remaining 192 positions is 23. Most of this growth happens in the last two of the six blocks, where the norm increases by almost the full integral of $\|h_d\|_C$. Since the per-step change of the norm is at most $\Delta t\, \|h_d\|_C$, with equality only when h_d is parallel to x_d, these blocks amplify the corner tokens along their own direction, and the tokens are still growing when the stage ends. Averaged over 500 images, the corner tokens still show large values in the same few channels with the same signs (Figure 15), which suggests a mechanism largely independent of the input. Averaging over positions instead (Figure 14, right), one channel grows above the rest in a block-wise sawtooth similar to that of the untied ConvNeXt, but its final spatial mean is only about three times that of the next largest channel.

Interpretation. Massive activations arise in pre-norm residual blocks and are suppressed when the vector-wise normalisation is replaced by an element-wise one (Sun et al., 2026). This fits what we see: ConvNeXt and Swin normalise each position across channels with LayerNorm, whereas BatchNorm (Ioffe & Szegedy, 2015) in ResNet normalises each channel independently; ResNet stage 3 shows no outlier. A channel-concentrated pattern has also been reported for diffusion transformers (Gan et al., 2025). In language models massive activations are injected by a few early blocks and then stay nearly constant until late blocks remove them, acting as fixed bias terms (Sun et al., 2024; 2026). The ConvNeXt channel instead evolves throughout the stage with the same kinematic profile as the rest of the state (Figure 13). In the shared model the explicit parameters are identical at every step, so the block can access depth only through the state. We conjecture that this channel provides that signal, as an autonomous system can emulate a non-autonomous one by carrying time as an extra state variable (Strogatz, 2015), which time-dependent Neural ODEs implement explicitly (Dupont et al., 2019). Our observations are consistent with this reading but do not establish it.

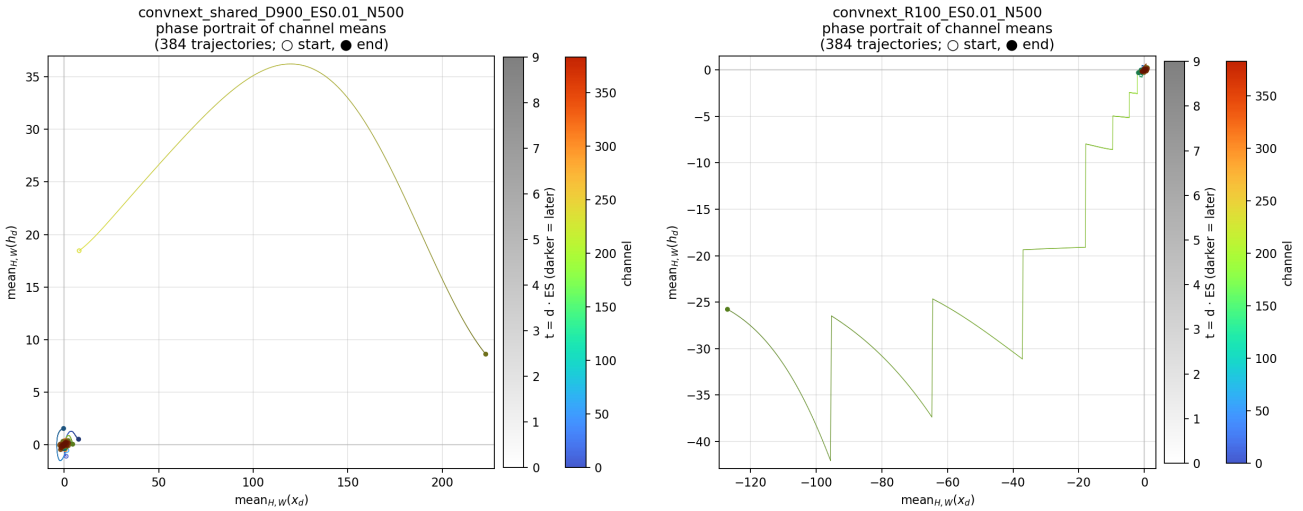


Figure 13. Phase portraits of stage-3 channel means $(\mathrm{mean}_{H,W}(x_d), \mathrm{mean}_{H,W}(h_d))$ over T=9 (Δt=0.01, N=500 images). **Left:** shared ConvNeXt (900 applications of the shared block). **Right:** untied ConvNeXt-T (R=100). One channel reaches spatial means more than an order of magnitude larger than those of the remaining 383 channels, which stay close to the origin.

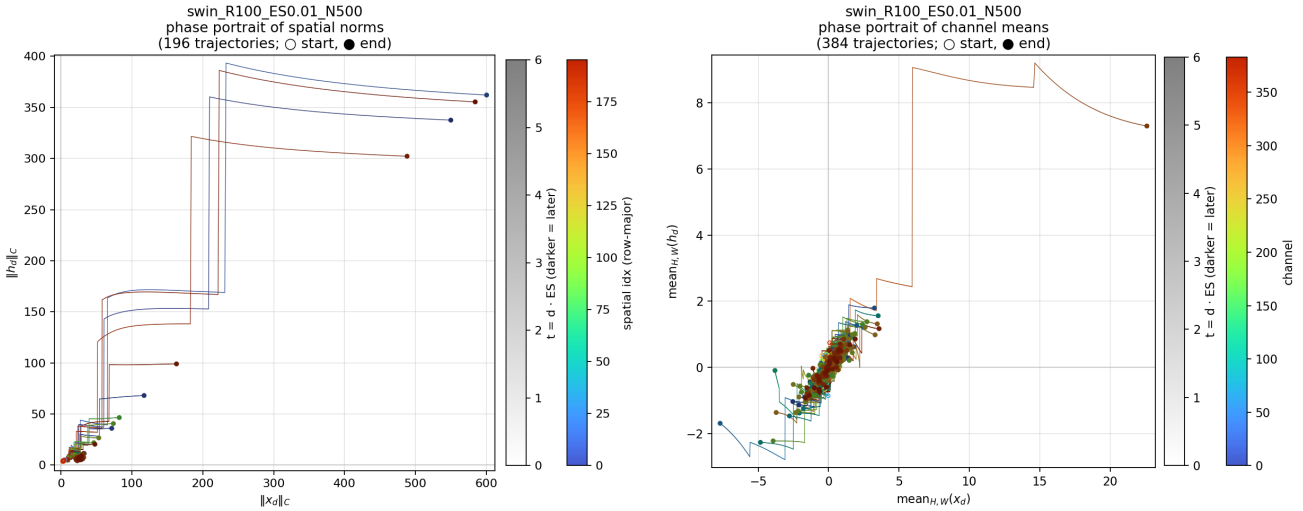


Figure 14. Swin-T stage 3 (R=100, Δt=0.01, T=6, N=500 images). **Left:** one trajectory per spatial position, $(\|x_d\|_C, \|h_d\|_C)$; the four outlying trajectories are the corner tokens. **Right:** one trajectory per channel, $(\mathrm{mean}_{H,W}(x_d), \mathrm{mean}_{H,W}(h_d))$; one channel grows above the rest in a block-wise sawtooth.

Open questions. Two tests would clarify the role of the ConvNeXt channel. Following the set-to-mean intervention of Sun et al. (2024), replacing it by its mean over images at the same step, instead of zero, would also approximately preserve the LayerNorm statistics and thus separate the effect of the channel from that of the renormalisation. Tracking the ratio between the channel and the RMS of the remaining channels along depth separates the two readings of Section 4.2: as an implicit parameter the ratio should stay roughly constant, whereas as a depth signal it should vary systematically with t, particularly in the shared model. A finer characterisation of the Swin corner tokens, including how they evolve after stage 3, is left for future work.

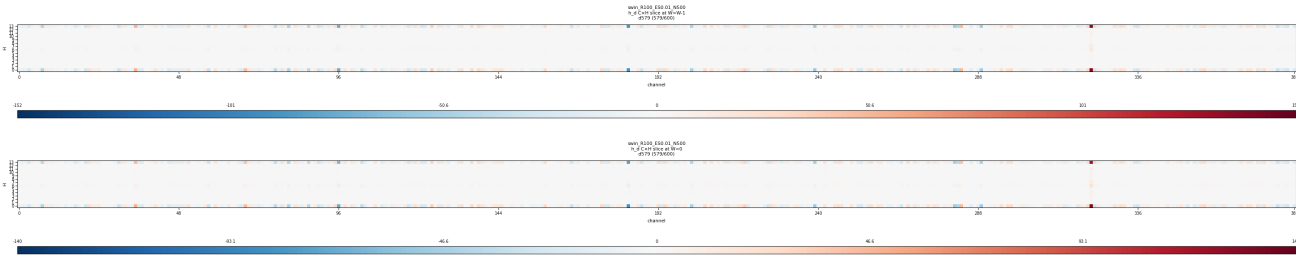


Figure 15. Residual h_d of Swin-T stage 3 at step d=579 (last block), averaged over 500 images and shown as channel by height slices at the last column (**top**) and at W=0 (**bottom**). Rows 0 and 13 of each panel are the four corner tokens, which share the same dominant channels and signs.

H. Per-coordinate statistics are set by the Gram matrix

An earlier version of this work summarised the stage-3 trajectories with statistics of individual feature coordinates: how often a coordinate changes sign between consecutive blocks, how much of its travel does not contribute to its net displacement and how often it reverses twice or more. Such statistics depend on the feature basis, which is not privileged: a rotation of the channels that is absorbed by the adjacent linear layers leaves the network's function unchanged and changes them. This appendix shows that, for these trajectories, they are almost entirely determined by the Gram matrix G of the block increments, so they carry no information beyond the basis-independent statistics of Section 4.3.

Statistics. For each image, with the massive channel removed, write $\Delta x_{k,i}$ for the change of coordinate i over the unit interval $[k, k{+}1]$ and $\mathrm{net}_i = \sum_k \Delta x_{k,i}$. *Excess travel* is $1 - \sum_i |\mathrm{net}_i| / \sum_{k,i} |\Delta x_{k,i}|$; the *monotone share* is the mass-weighted share of coordinates whose nine increments all have the same sign; the *sign-change rate* is the share of consecutive pairs $(k, k{+}1)$ with opposite signs; the *reversed mass* adds $\min(|\Delta x_{k,i}|, |\Delta x_{k+1,i}|)$ over those pairs, normalised by $\sum_{k,i} |\Delta x_{k,i}|$; and the *repeated sign-change share* is the mass-weighted share of coordinates that change sign at least twice. All statistics use every coordinate; the earlier text described a selection of the coordinates whose net displacement is above the per-image median, but the code applied it only to the median t_{50} of the per-coordinate progress.

Surrogates. We replace the real increments by two surrogates that keep G and discard everything else: Gaussian increments with the same Gram matrix, and the real increments after a random orthogonal rotation of the channels (three seeds each). Table 17 shows that both reproduce every statistic within 0.03 on the untied stage (at R=1 and R=100) and within 0.08 on the tied one. Tied coordinates are slightly more monotone than their surrogates, the only basis-dependent structure we find. A sign-change rate of 0.45 is thus what a consecutive cosine of 0.17 implies in any basis, and the random stage, at cosine 0, has a higher one. Per spatial position the picture is the same: the cosine between consecutive untied increments restricted to one position has a 10th percentile of 0.03 at R=1, and no position has a cosine below -0.3.

Gaussian prediction. If two increments are jointly Gaussian across coordinates with correlation c, a coordinate changes sign with probability $\arccos(c)/\pi$. With the measured consecutive cosine, this predicts a sign-change rate of 0.447 for the untied stage at R=1, against 0.447 measured, and 0.217 for the tied stage, against 0.183 measured; the prediction and the measurement correlate at 0.92 to 0.96 across images and block pairs, and random blocks give 0.500.

A nuance on the rotation argument. A literal channel-wise rotation absorbed by the adjacent linear layers would leave the network's function unchanged and would change the per-coordinate statistics. ConvNeXt is not quite that object: the depthwise convolution, the per-channel LayerScale and the gains of LayerNorm all act per channel, and the massive activation shows that the channel basis is partly privileged (Appendix G). The empirical result nevertheless stands: Gaussian and randomly rotated surrogates that keep G reproduce the per-coordinate statistics to the tolerances above, so those statistics add nothing beyond the Gram geometry of Section 4.3.

Pooled coordinates. Figure 16 shows the per-coordinate progress $p_i(t) = (x_i(t) - x_i(0))/\mathrm{net}_i$ pooled over images. The tied coordinates advance mostly monotonically, while the untied ones change course at block boundaries: across one boundary 44.6% of the coordinates of the velocity change sign, against 0.19% per micro-step inside a block. By the argument above, this is the per-coordinate trace of nearly orthogonal increments, not evidence that later blocks remove earlier ones; Section 4.3 shows that they do not. Script: `scripts/undo_geometry_audit.py`; outputs in `outputs/undo_audit`.

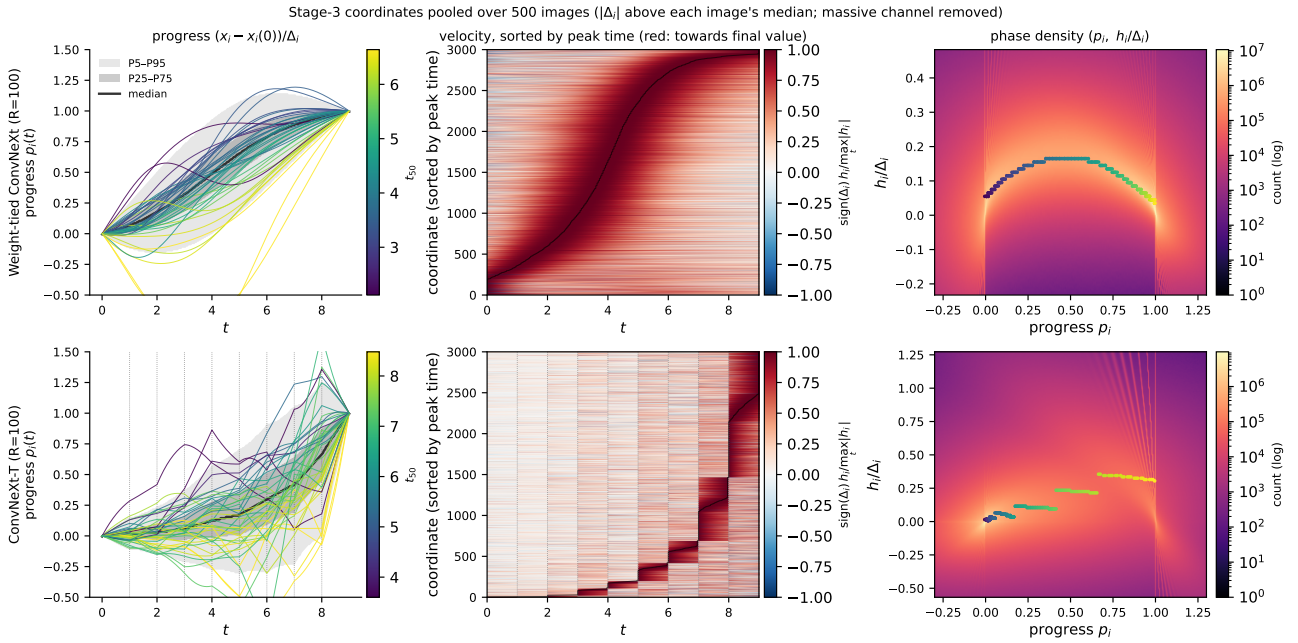


Figure 16. Stage-3 coordinates pooled over 500 images (R=100; massive channel removed). **Top:** weight-tied ConvNeXt; **bottom:** untied ConvNeXt-T. **Left:** per-coordinate progress $p_i(t)$ (median and P5–P95; thin lines: random coordinates). **Middle:** velocity toward the final value, coordinates sorted by peak time (red: toward the end state; blue: away). **Right:** phase density $(p_i,\ h_i/\mathrm{net}_i)$.

Table 17. Per-coordinate statistics of stage 3 at integer times (R=100, 500 images, massive channel removed) for the real increments and for Gaussian increments with the same Gram matrix (mean over three seeds; std ≤ 0.001).

Stage	source	excess	monotone	sign change	reversed	repeated
Random (g^*)	real	0.671	0.046	0.500	0.294	0.966
Untied	real	0.477	0.230	0.446	0.249	0.902
Untied	same Gram	0.490	0.205	0.445	0.262	0.905
Tied	real	0.188	0.739	0.114	0.018	0.137
Tied	same Gram	0.215	0.689	0.123	0.020	0.153

Table 18. Stage-3 Gram statistics at R=1 (mean over 500 images). s_k<0: share of image and block pairs with negative survival; E: opposed fraction.

Model	init	Str.	$\cos_{k,k+1}$	$\overline{\cos}$	min s_k	s_k<0	E	$1/\sqrt{n}$
Swin-T	trained	0.800	0.427	0.276	0.40	0%	0	0.408
Swin-T, no corners	trained	0.668	0.319	0.209	0.12	0%	0	0.408
Swin-T	g=1	0.417	0.020	0.008	−0.14	33%	0.012	0.408
Swin-T	g^*=1.8	0.418	0.016	0.009	−0.14	37%	0.012	0.408
ResNet-50	trained	0.371	−0.110	−0.080	−0.38	100%	0.187	0.447
ResNet-50	g=1	0.428	−0.036	−0.020	−0.18	75%	0.077	0.447
ResNet-50	g^*=1.15	0.430	−0.038	−0.021	−0.21	75%	0.088	0.447
ResNet-101	trained	0.158	0.009	−0.022	−0.96	89%	0.244	0.213
ResNet-101	g=1	0.229	−0.032	0.004	−0.30	50%	0.070	0.213
ResNet-101	g^*=0.95	0.221	−0.029	0.002	−0.32	60%	0.069	0.213

I. Transport coherence in other architectures

We compute the Gram statistics of Section 4.3 at native resolution (R=1) on the same 500 images for stage 3 of the torchvision Swin-T, ResNet-50 and ResNet-101 and for random stages with Xavier weights at gain 1 and at the gain g^* that matches the trained field strength (Appendix D). Swin is reported with all tokens and without the four corner tokens (Appendix G). ResNet stages exclude the first block, which has a projection shortcut, and the field f_i includes the ReLU applied after the sum, so n=5 and n=22. Nine, six or twenty-two orthogonal increments of equal length would give a straightness of $1/\sqrt{n}$ (last column).

Swin-T stage 3 is a coherent transport: straightness 0.80, consecutive cosine 0.43 and positive survival for every image and block. Without the corner tokens it is still well above its random baseline (0.67 against 0.42), so the coherence is not only the growth of the corner tokens along their own direction. Its random baseline sits at $1/\sqrt{6}$, as for ConvNeXt.

Both ResNets go the other way and are the only architectures here with genuine reversal: opposed fraction E is 0.19 (ResNet-50) and 0.24 (ResNet-101), against about 0.07–0.09 for their random baselines, while ConvNeXt and Swin have E=0. Trained ResNet-50 is *less* straight than its random baseline (0.37 against 0.43), with negative consecutive cosine (−0.11) and negative survival for every image and block; ResNet-101 behaves similarly (0.16 against 0.22 to 0.23). Random ResNet stages already show negative survival, unlike random ConvNeXt stages. One candidate is the ReLU after the sum, which makes f_i include a rectification of $x + F_i(x)$ that can remove part of x; we have not tested it, and we report the ResNet statistics as a fact

about these checkpoints rather than as a mechanism. Script: `scripts/transport_coherence_arch.py`; outputs in `outputs/transport_coherence_arch`.